UNIT I

The software is an instruction or computer program that when executed provides desired features, function, and performance

Characteristics of software:

There is some characteristic of software which is given below:

1. **Reliability:** The ability of the software to consistently perform its intended tasks without unexpected failures or errors.
2. **Usability:** How easily and effectively users can interact with and navigate through the software.
3. **Efficiency:** The optimal utilization of system resources to perform tasks on time.
4. **Maintainability:** How easily and cost-effectively software can be modified, updated, or extended.
5. **Portability:** The ability of software to run on different platforms or environments without requiring significant modifications.

Nature of Software:

1. **System Software:** System software is a collection of programs that are written to service other programs. Some system software processes complex but determinate, information structures. Other system application processes largely indeterminate data. Sometimes when, the system software area is characterized by the heavy interaction with computer hardware that requires scheduling, resource sharing, and sophisticated process management.
2. **Application Software:** Application software is defined as programs that solve a specific business need. Application in this area processes business or technical data in a way that facilitates business operation or management technical decision-making. In addition to conventional data processing applications, application software is used to control business functions in real-time.
3. **Engineering and Scientific Software:** This software is used to facilitate the engineering function and task. however modern applications within the engineering and scientific area are moving away from conventional numerical algorithms. Computer-aided design, system simulation, and other interactive applications have begun to take a real-time and even system software characteristic.
4. **Embedded Software:** Embedded software resides within the system or product and is used to implement and control features and functions for the end-user and for the system itself. Embedded software can perform limited and esoteric functions or provide significant function and control capability.
5. **Product-line Software:** Designed to provide a specific capability for use by many customers, product-line software can focus on the limited and esoteric marketplace or address the mass consumer market.
6. **Web Application:** It is a client-server computer program that the client runs on the web browser. In their simplest form, Web apps can be little more than a set of linked hypertext files that present information using text and limited graphics. However, as e-commerce and B2B applications grow in importance. Web apps are evolving into a sophisticated computing environment that not only provides a standalone feature, computing function, and content to the end user.
7. **Artificial Intelligence Software:** Artificial intelligence software makes use of a nonnumerical algorithm to solve a complex problem that is not amenable to computation or straightforward analysis. Applications within this area include robotics, expert systems, pattern recognition, artificial neural networks, theorem proving, and game playing.

What is Software Engineering?

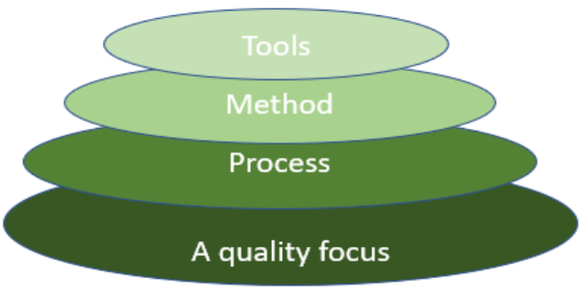
Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

Software Engineering

1. Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.
2. It is a rapidly evolving field, and new tools and technologies are constantly being developed to improve the software development process.
3. By following the principles of software engineering and using the appropriate tools and methodologies, software developers can create high-quality, reliable, and maintainable software that meets the needs of its users.
4. Software Engineering is mainly used for large projects based on software systems rather than single programs or applications.
5. The main goal of Software Engineering is to develop software applications for improving quality, budget, and time efficiency.
6. Software Engineering ensures that the software that has to be built should be consistent, correct, also on budget, on time, and within the required requirements.

Layered Technology

Software engineering is a fully layered technology, to develop software we need to go from one layer to another. All the layers are connected and each layer demands the fulfillment of the previous layer.



Layered technology is divided into four parts:

1. A quality focus: It defines the continuous process improvement principles of software. It provides integrity that means providing security to the software so that data can be accessed by only an authorized person, no outsider can access the data. It also focuses on maintainability and usability.

2. Process: It is the foundation or base layer of software engineering. It is key that binds all the layers together which enables the development of software before the deadline or on time. Process defines a framework that must be established for the effective delivery of software engineering technology. The software process covers all the activities, actions, and tasks required to be carried out for software development.

Process activities are listed below:-

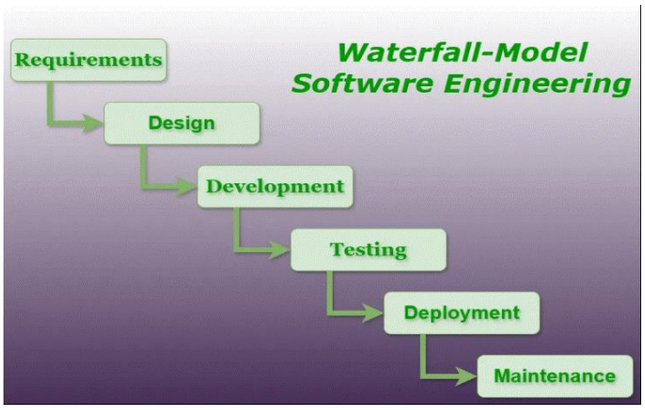
- **Communication:** It is the first and foremost thing for the development of software. Communication is necessary to know the actual demand of the client.
- **Planning:** It basically means drawing a map for reduced the complication of development.
- **Modelling:** In this process, a model is created according to the client for better understanding.
- **Construction:** It includes the coding and testing of the problem.
- **Deployment:-** It includes the delivery of software to the client for evaluation and feedback.

3. Method: During the process of software development the answers to all "how-to-do" questions are given by method. It has the information of all the tasks which includes communication, requirement analysis, design modeling, program construction, testing, and support.

4. Tools: Software engineering tools provide a self-operating system for processes and methods. Tools are integrated which means information created by one tool can be used by another.

What is the SDLC Waterfall Model?

The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to project management and software development.
The waterfall model is useful in situations where the project requirements are well-defined and the project goals are clear. It is often used for large-scale projects with long timelines, where there is little room for error and the project stakeholders need to have a high level of confidence in the outcome.



Features of Waterfall Model

Following are the features of the waterfall model:

1. **Sequential Approach**: The waterfall model involves a sequential approach to software development, where each phase of the project is completed before moving on to the next one.
2. **Document-Driven:** The waterfall model depended on documentation to ensure that the project is well-defined and the project team is working towards a clear set of goals.
3. **Quality Control:** The waterfall model places a high emphasis on quality control and testing at each phase of the project, to ensure that the final product meets the requirements and expectations of the stakeholders.
4. **Rigorous Planning**: The waterfall model involves a careful planning process, where the project scope, timelines, and deliverables are carefully defined and monitored throughout the project lifecycle.

Phases of Waterfall Model

The Waterfall Model has six phases which are:

1. **Requirements:** The first phase involves gathering requirements from stakeholders and analyzing them to understand the scope and objectives of the project.
2. **Design:** Once the requirements are understood, the design phase begins. This involves creating a detailed design document that outlines the software architecture, user interface, and system components.
3. **Development:** The Development phase include implementation involves coding the software based on the design specifications. This phase also includes unit testing to ensure that each component of the software is working as expected.
4. **Testing:** In the testing phase, the software is tested as a whole to ensure that it meets the requirements and is free from defects.
5. **Deployment:** Once the software has been tested and approved, it is deployed to the production environment.
6. **Maintenance:** The final phase of the Waterfall Model is maintenance, which involves fixing any issues that arise after the software has been deployed and ensuring that it continues to meet the requirements over time.

The classical waterfall model divides the life cycle into a set of phases. This model considers that one phase can be started after the completion of the previous phase. That is the output of one phase will be the input to the next phase. Thus the development

process can be considered as a sequential flow in the waterfall. Here the phases do not overlap with each other.

When to use SDLC Waterfall Model?

Some Circumstances where the use of the Waterfall model is most suited are:

- When the requirements are constant and not changed regularly.
- A project is short
- The situation is calm
- Where the tools and technology used is consistent and is not changing
- When resources are well prepared and are available to use.

Advantages of Waterfall model

- This model is simple to implement also the number of resources that are required for it is minimal.
- The requirements are simple and explicitly declared; they remain unchanged during the entire project development.
- The start and end points for each phase is fixed, which makes it easy to cover progress.
- The release date for the complete product, as well as its final cost, can be determined before development.
- It gives easy to control and clarity for the customer due to a strict reporting system.

Disadvantages of Waterfall model

- In this model, the risk factor is higher, so this model is not suitable for more significant and complex projects.
- This model cannot accept the changes in requirements during development.
- It becomes tough to go back to the phase. For example, if the application has now shifted to the coding phase, and there is a change in requirement, It becomes tough to go back and change it.
- Since the testing done at a later stage, it does not allow identifying the challenges and risks in the earlier phase, so the risk reduction strategy is difficult to prepare.

Incremental Model

Incremental Model is a process of software development where requirements divided into multiple standalone modules of the software development cycle. In this model, each module goes through the requirements, design, implementation and testing phases. Every subsequent release of the module adds function to the previous release. The process continues until the complete system achieved.

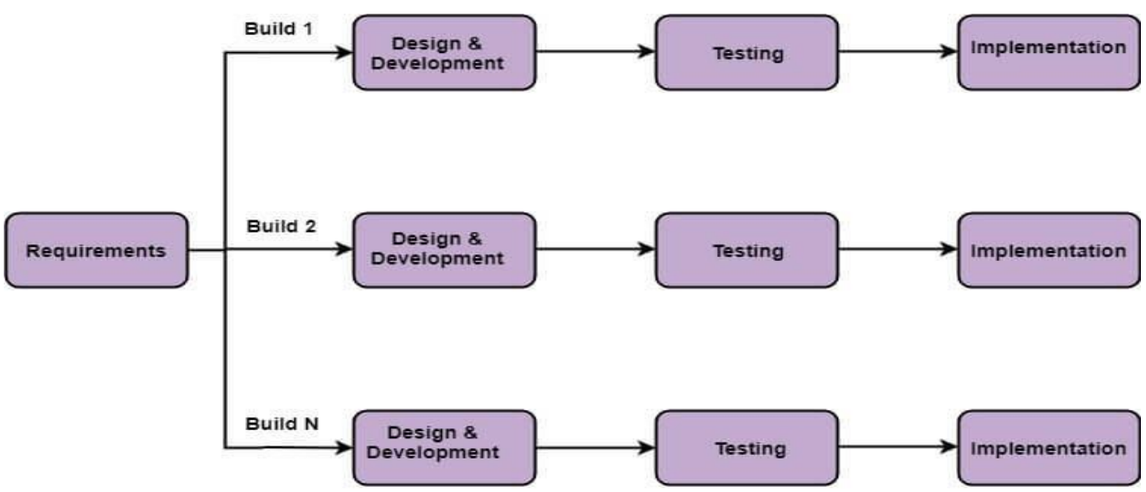


Fig: Incremental Model

The various phases of incremental model are as follows:

1. Requirement analysis: In the first phase of the incremental model, the product analysis expertise identifies the requirements. And the system functional requirements are understood by the requirement analysis team. To develop the software under the incremental model, this phase performs a crucial role.

2. Design & Development: In this phase of the Incremental model of SDLC, the design of the system functionality and the development method are finished with success. When software develops new practicality, the incremental model uses style and development phase.

3. Testing: In the incremental model, the testing phase checks the performance of each existing function as well as additional functionality. In the testing phase, the various methods are used to test the behavior of each task.

4. Implementation: Implementation phase enables the coding phase of the development system. It involves the final coding that design in the designing and development phase and tests the functionality in the testing phase. After completion of this phase, the number of the product working is enhanced and upgraded up to the final system product

When we use the Incremental Model?

- When the requirements are superior.
- A project has a lengthy development schedule.
- When Software team are not very well skilled or trained.
- When the customer demands a quick release of the product.
- You can develop prioritized requirements first.

Advantage of Incremental Model

- Errors are easy to be recognized.
- Easier to test and debug
- More flexible.
- Simple to manage risk because it handled during its iteration.
- The Client gets important functionality early.

Disadvantage of Incremental Model

- Need for good planning
- Total Cost is high.
- Well defined module interfaces are needed.

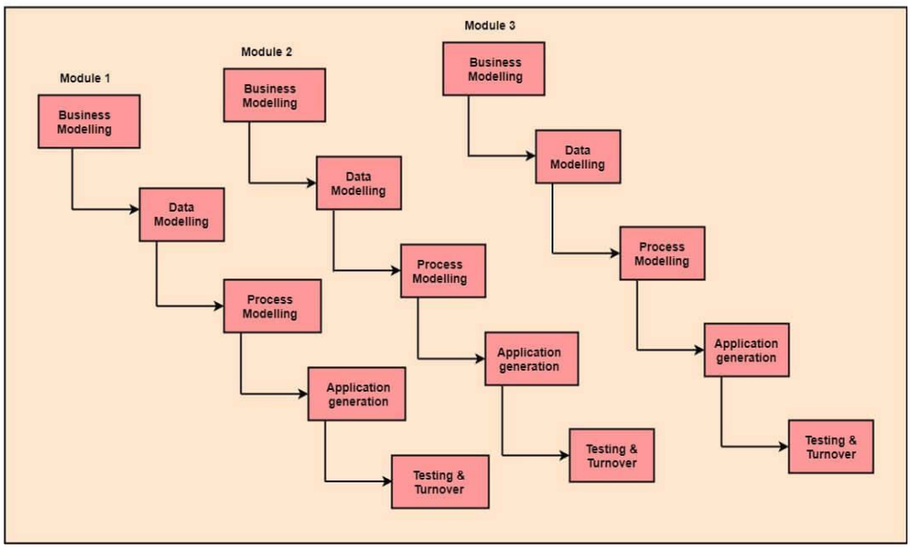
RAD (Rapid Application Development) Model

RAD is a linear sequential software development process model that emphasizes a concise development cycle using an element based construction approach. If the requirements are well understood and described, and the project scope is a constraint, the RAD process enables a development team to create a fully functional system within a concise time period.

RAD (Rapid Application Development) is a concept that products can be developed faster and of higher quality through:

- Gathering requirements using workshops or focus groups
- Prototyping and early, reiterative user testing of designs
- The re-use of software components
- A rigidly paced schedule that refers design improvements to the next product version
- Less formality in reviews and other team communication

Fig: RAD Model



The various phases of RAD are as follows:

1.Business Modelling: The information flow among business functions is defined by answering questions like what data drives the business process, what data is generated, who generates it, where does the information go, who process it and so on.

2. Data Modelling: The data collected from business modeling is refined into a set of data objects (entities) that are needed to support the business. The attributes (character of each entity) are identified, and the relation between these data objects (entities) is defined.

3. Process Modelling: The information object defined in the data modeling phase are transformed to achieve the data flow necessary to implement a business function. Processing descriptions are created for adding, modifying, deleting, or retrieving a data object.

4. Application Generation: Automated tools are used to facilitate construction of the software; even they use the 4th GL techniques.

5. Testing & Turnover: Many of the programming components have already been tested since RAD emphasis reuse. This reduces the overall testing time. But the new part must be tested, and all interfaces must be fully exercised.

When to use RAD Model?

- When the system should need to create the project that modularizes in a short span time (2-3 months).
- When the requirements are well-known.
- When the technical risk is limited.
- When there's a necessity to make a system, which modularized in 2-3 months of period.
- It should be used only if the budget allows the use of automatic code generating tools.

Advantage of RAD Model

- This model is flexible for change.
- In this model, changes are adoptable.
- Each phase in RAD brings highest priority functionality to the customer.
- It reduced development time.
- It increases the reusability of features.

Disadvantage of RAD Model

- It required highly skilled designers.
- All application is not compatible with RAD.
- For smaller projects, we cannot use the RAD model.
- On the high technical risk, it's not suitable.
- Required user involvement.

Prototype Model

The prototype model requires that before carrying out the development of actual software, a working prototype of the system should be built. A prototype is a toy implementation of the system. A prototype usually turns out to be a very crude version of the actual system, possible exhibiting limited functional capabilities, low reliability, and inefficient performance as compared to actual software. In many instances, the client only has a general view of what is expected from the software product. In such a scenario where there is an absence of detailed information regarding the input to the system, the processing needs, and the output requirement, the prototyping model may be employed.

Steps of Prototype Model

1. Requirement Gathering and Analyst
2. Quick Decision
3. Build a Prototype
4. Assessment or User Evaluation
5. Prototype Refinement
6. Engineer Product

Advantage of Prototype Model

1. Reduce the risk of incorrect user requirement
2. Good where requirement are changing/uncommitted
3. Regular visible process aids management
4. Support early product marketing
5. Reduce Maintenance cost.
6. Errors can be detected much earlier as the system is made side by side.

Disadvantage of Prototype Model

1. An unstable/badly implemented prototype often becomes the final product.
2. Require extensive customer collaboration
    - Costs customer money
    - Needs committed customer
    - Difficult to finish if customer withdraw
    - May be too customer specific, no broad market
3. Difficult to know how long the project will last.
4. Easy to fall back into the code and fix without proper requirement analysis, design, customer evaluation, and feedback.
5. Prototyping tools are expensive.
6. Special tools & techniques are required to build a prototype.
7. It is a time-consuming process.

Spiral Model

The spiral model, initially proposed by Boehm, is an evolutionary software process model that couples the iterative feature of prototyping with the controlled and systematic aspects of the linear sequential model. It implements the potential for rapid development of new versions of the software. Using the spiral model, the software is developed in a series of incremental releases. During the early iterations, the additional release may be a paper model or prototype. During later iterations, more and more complete versions of the ngineered system are produced.

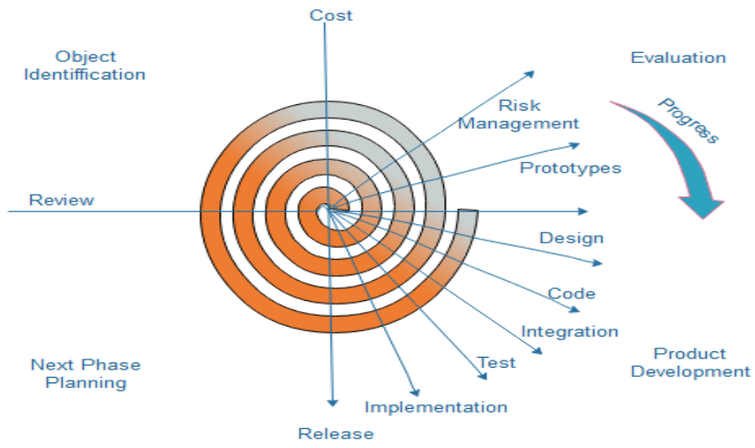


Fig. Spiral Model

Each cycle in the spiral is divided into four parts:

Objective setting: Each cycle in the spiral starts with the identification of purpose for that cycle, the various alternatives that are possible for achieving the targets, and the constraints that exists.

Risk Assessment and reduction: The next phase in the cycle is to calculate these various alternatives based on the goals and constraints. The focus of evaluation in this stage is located on the risk perception for the project.

Development and validation: The next phase is to develop strategies that resolve uncertainties and risks. This process may include activities such as benchmarking, simulation, and prototyping.

Planning: Finally, the next step is planned. The project is reviewed, and a choice made whether to continue with a further period of the spiral. If it is determined to keep, plans are drawn up for the next step of the project.

The development phase depends on the remaining risks. For example, if performance or user-interface risks are treated more essential than the program development risks, the next phase may be an evolutionary development that includes developing a more detailed prototype for solving the risks.

The **risk-driven** feature of the spiral model allows it to accommodate any mixture of a specification-oriented, prototype-oriented, simulation-oriented, or another type of approach. An essential element of the model is that each period of the spiral is completed by a review that includes all the products developed during that cycle, including plans for the next cycle. The spiral model works for development as well as enhancement projects.

When to use Spiral Model?

- When deliverance is required to be frequent.
- When the project is large
- When requirements are unclear and complex
- When changes may require at any time
- Large and high budget projects

Advantages

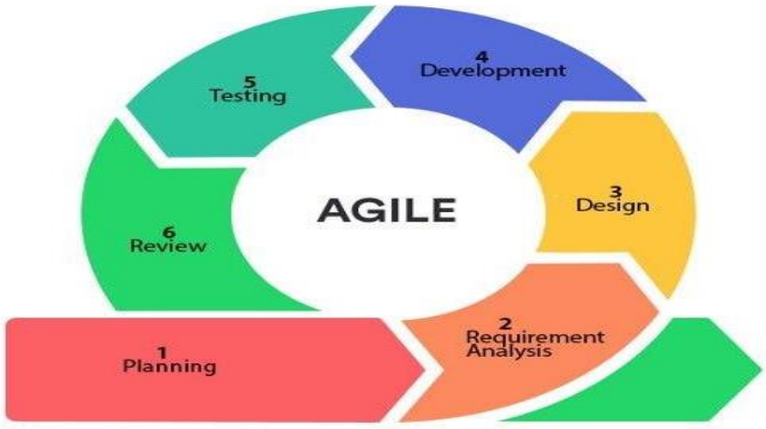
- High amount of risk analysis
- Useful for large and mission-critical projects.

Disadvantages

- Can be a costly model to use.
- Risk analysis needed highly particular expertise
- Doesn't work well for smaller projects.

Agile Model

The meaning of Agile is swift or versatile. **"Agile process model"** refers to a software development approach based on iterative development. Agile methods break tasks into smaller iterations, or parts do not directly involve long term planning. The project scope and requirements are laid down at the beginning of the development process. Plans regarding the number of iterations, the duration and the scope of each iteration are clearly defined in advance.



Phases of Agile Model:

1. Requirements gathering: In this phase, you must define the requirements. You should explain business opportunities and plan the time and effort needed to build the project. Based on this information, you can evaluate technical and economic feasibility.
2. Design the requirements: When you have identified the project, work with stakeholders to define requirements. You can use the user flow diagram or the high-level UML diagram to show the work of new features and show how it will apply to your existing system.
3. Construction/ iteration: When the team defines the requirements, the work begins. Designers and developers start working on their project, which aims to deploy a working product. The product will undergo various stages of improvement, so it includes simple, minimal functionality.
4.Testing: In this phase, the Quality Assurance team examines the product's performance and looks for the bug.
5. Deployment: In this phase, the team issues a product for the user's work environment.
6. Feedback: After releasing the product, the last step is feedback. In this, the team receives feedback about the product and works through the feedback.

When to use the Agile Model?

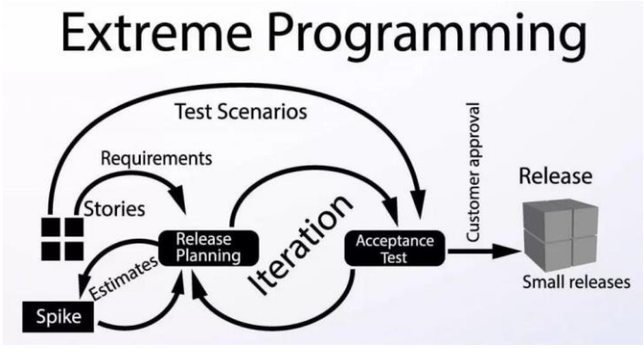
- When frequent changes are required.
- When a highly qualified and experienced team is available.
- When a customer is ready to have a meeting with a software team all the time.
- When project size is small.

Advantage of Agile Method:

1. Frequent Delivery
2. Face-to-Face Communication with clients.
3. Efficient design and fulfils the business requirement.
4. Anytime changes are acceptable.
5. It reduces total development time.

Extreme Programming (XP)

Extreme Programming (XP) is an Agile software development methodology that focuses on delivering high-quality software through frequent and continuous feedback, collaboration, and adaptation. XP emphasizes a close working relationship between the development team, the customer, and stakeholders, with an emphasis on rapid, iterative development and deployment.

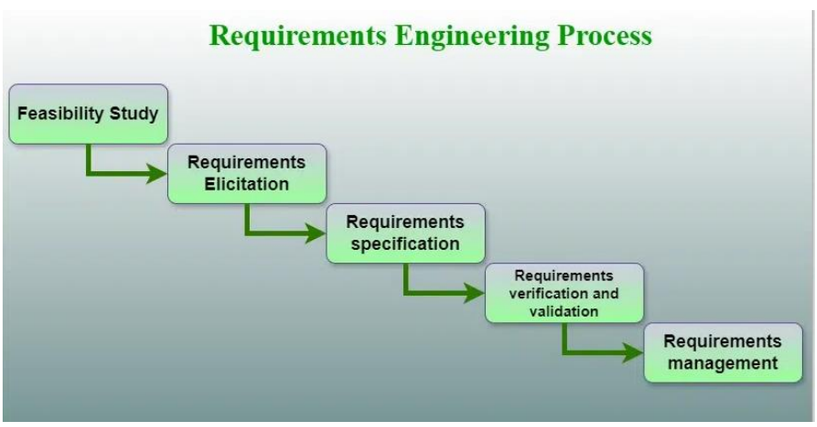


Good Practices in Extreme Programming

- **Code Review:** Code review detects and corrects errors efficiently. It suggests pair programming as coding and reviewing of written code carried out by a pair of programmers who switch their work between them every hour.
- **Testing:** Testing code helps to remove errors and improves its reliability. XP suggests test-driven development (TDD) to continually write and execute test cases. In the TDD approach, test cases are written even before any code is written.
- **Incremental development:** Incremental development is very good because customer feedback is gained and based on this development team comes up with new increments every few days after each iteration.
- **Simplicity:** Simplicity makes it easier to develop good-quality code as well as to test and debug it.
- **Design:** Good quality design is important to develop good quality software. So, everybody should design daily.
- **Integration testing:** Integration Testing helps to identify bugs at the interfaces of different functionalities. Extreme programming suggests that the developers should achieve continuous integration by building and performing integration testing several times a day.

Requirements Engineering

A systematic and strict approach to the definition, creation, and verification of requirements for a software system is known as requirements engineering. To guarantee the effective creation of a software product, the requirements engineering process entails several tasks that help in understanding, recording, and managing the demands of stakeholders.



1. **Feasibility Study**
2. **Requirements elicitation**
3. **Requirements specification**
4. **Requirements for verification and validation**
5. **Requirements management**

1. Feasibility Study:

The objective behind the feasibility study is to create the reasons for developing the software that is acceptable to users, flexible to change and conformable to established standards.

Types of Feasibility:

1. **Technical Feasibility** - Technical feasibility evaluates the current technologies, which are needed to accomplish customer requirements within the time and budget.
2. **Operational Feasibility** - Operational feasibility assesses the range in which the required software performs a series of levels to solve business problems and customer requirements.
3. **Economic Feasibility** - Economic feasibility decides whether the necessary software can generate financial profits for an organization.

2. Requirement Elicitation and Analysis:

This is also known as the **gathering of requirements**. Here, requirements are identified with the help of customers and existing systems processes, if available.
Analysis of requirements starts with requirement elicitation. The requirements are analyzed to identify inconsistencies, defects, omission, etc. We describe requirements in terms of relationships and also resolve conflicts if any.

Problems of Elicitation and Analysis

- Getting all, and only, the right people involved.
- Stakeholders often don't know what they want
- Stakeholders express requirements in their terms.
- Stakeholders may have conflicting requirements.
- Requirement change during the analysis process.
- Organizational and political factors may influence system requirements.

3. Software Requirement Specification:

Software requirement specification is a kind of document which is created by a software analyst after the requirements collected from the various sources - the requirement received by the customer written in ordinary language. It is the job of the analyst to write the requirement in technical language so that they can be understood and beneficial by the development team.
The models used at this stage include ER diagrams, data flow diagrams (DFDs), function decomposition diagrams (FDDs), data dictionaries, etc.

- **Data Flow Diagrams:** Data Flow Diagrams (DFDs) are used widely for modeling the requirements. DFD shows the flow of data through a system. The system may be a company, an organization, a set of procedures, a computer hardware system, a software system, or any combination of the preceding. The DFD is also known as a data flow graph or bubble chart.
- **Data Dictionaries:** Data Dictionaries are simply repositories to store information about all data items defined in DFDs. At the requirements stage, the data dictionary should at least define customer data items, to ensure that the customer and developers use the same definition and terminologies.
- **Entity-Relationship Diagrams:** Another tool for requirement specification is the entity-relationship diagram, often called an "***E-R diagram***." It is a detailed logical representation of the data for the organization and uses three main constructs i.e. data entities, relationships, and their associated attributes.

4. Software Requirement Validation:

After requirement specifications developed, the requirements discussed in this document are validated. The user might demand illegal, impossible solution or experts may misinterpret the needs. Requirements can be the check against the following conditions

- If they can practically implement
- If they are correct and as per the functionality and specially of software
- If there are any ambiguities
- If they are full

- If they can describe

Requirements Validation Techniques

- **Requirements reviews/inspections:** systematic manual analysis of the requirements.
- **Prototyping:** Using an executable model of the system to check requirements.
- **Test-case generation:** Developing tests for requirements to check testability.
- **Automated consistency analysis:** checking for the consistency of structured requirements descriptions.

5. Software Requirement Management:

Requirement management is the process of managing changing requirements during the requirements engineering process and system development. New requirements emerge during the process as business needs a change, and a better understanding of the system is developed. The priority of requirements from different viewpoints changes during development process. The business and technical environment of the system changes during the development.

Requirements Elicitation

Requirements elicitation is the process of gathering and defining the requirements for a software system. The goal of requirements elicitation is to ensure that the software development process is based on a clear and comprehensive understanding of the customer's needs and requirements.

What is Requirement Elicitation?

The process of investigating and learning about a system's requirements from users, clients, and other stakeholders is known as requirements elicitation. Requirements elicitation in software engineering is perhaps the most difficult, most error-prone, and most communication-intensive software development.

1. Requirement Elicitation can be successful only through an effective customer-developer partnership. It is needed to know what the users require.
2. Requirements elicitation involves the identification, collection, analysis, and refinement of the requirements for a software system.
3. Requirement Elicitation is a critical part of the software development life cycle and is typically performed at the beginning of the project.
4. Requirements elicitation involves stakeholders from different areas of the organization, including business owners, end-users, and technical experts.
5. The output of the requirements elicitation process is a set of clear, concise, and well-defined requirements that serve as the basis for the design and development of the software system.
6. Requirements elicitation is difficult because just questioning users and customers about system needs may not collect all relevant requirements, particularly for safety and dependability.
7. Interviews, surveys, user observation, workshops, brainstorming, use cases, role-playing, and prototyping are all methods for eliciting requirements.

Software Requirement Specifications

The production of the requirements stage of the software development process is **Software Requirements Specifications (SRS)** (also called a **requirements document**). This report lays a foundation for software engineering activities and is constructing when entire requirements are elicited and analysed. **SRS** is a formal report, which acts as a representation of software that enables the customers to review whether it (SRS) is according to their requirements. Also, it comprises user requirements for a system as well as detailed specifications of the system requirements.

The SRS is a specification for a specific software product, program, or set of applications that perform particular functions in a specific environment. It serves several goals depending on who is writing it. First, the SRS could be written by the client of a system. Second, the SRS could be written by a developer of the system. The two methods create entirely various situations and establish different purposes for the document altogether. The first case, SRS, is used to define the needs and expectation of the users.

The second case, SRS, is written for various purposes and serves as a contract document between customer and developer.

Software Requirement Specification (SRS) Format as the name suggests, is a complete specification and description of requirements of the software that need to be fulfilled for the successful development of the software system. These requirements can be functional as well as non-functional depending upon the type of requirement. The interaction between different customers and contractors is done because it is necessary to understand the needs of customers Depending upon information gathered after interaction, SRS is developed which describes requirements of software that may include changes and modifications that is needed to be done to increase quality of product and to satisfy customer's demand

Document Title

Author(s)

Affiliation

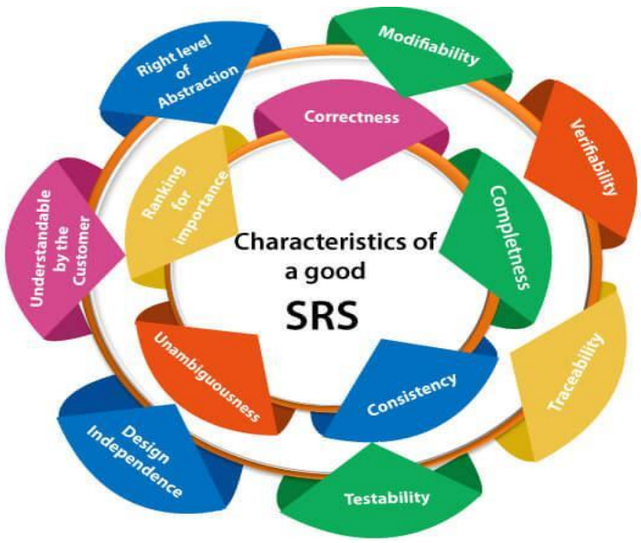
Address

Date

Document Version

.

Characteristics of good SRS



Following are the features of a good SRS document:

1. Correctness: User review is used to provide the accuracy of requirements stated in the SRS. SRS is said to be perfect if it covers all the needs that are truly expected from the system.

2. Completeness: The SRS is complete if, and only if, it includes the following elements:

- All essential requirements, whether relating to functionality, performance, design, constraints, attributes, or external interfaces.
- Definition of their responses of the software to all realizable classes of input data in all available categories of situations.
- Full labels and references to all figures, tables, and diagrams in the SRS and definitions of all terms and units of measure.

3. Consistency: The SRS is consistent if, and only if, no subset of individual requirements described in its conflict.

4. Unambiguousness: SRS is unambiguous when every fixed requirement has only one interpretation. This suggests that each element is uniquely interpreted. In case there is a method used with multiple definitions, the requirements report should determine the implications in the SRS so that it is clear and simple to understand.

5. Ranking for importance and stability: The SRS is ranked for importance and stability if each requirement in it has an identifier to indicate either the significance or stability of that particular requirement.

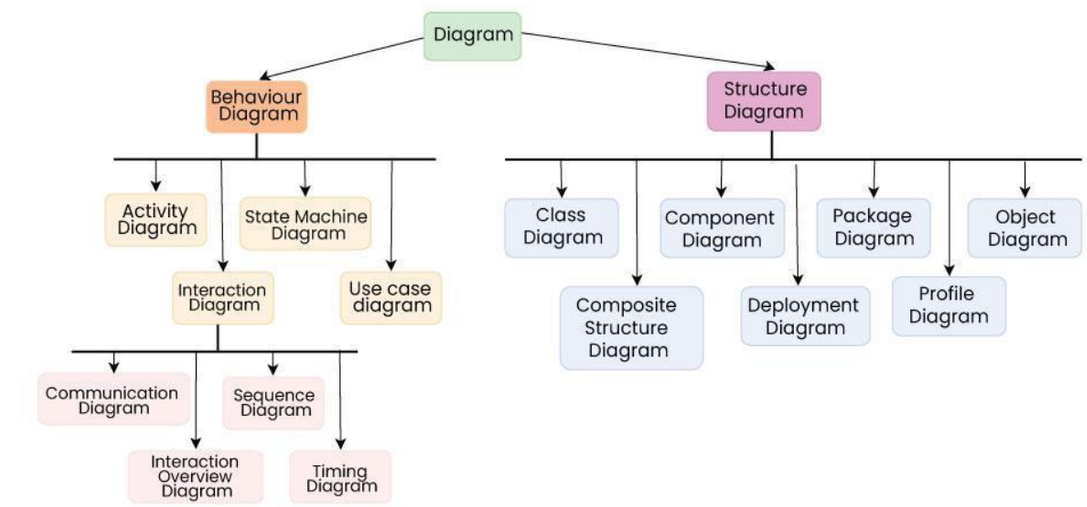
Typically, all requirements are not equally important. Some prerequisites may be essential, especially for life-critical applications, while others may be desirable. Each

element should be identified to make these differences clear and explicit. Another way to rank requirements is to distinguish classes of items as essential, conditional, and optional.
6. Modifiability: SRS should be made as modifiable as likely and should be capable of quickly obtain changes to the system to some extent. Modifications should be perfectly indexed and cross-referenced.
7. Verifiability: SRS is correct when the specified requirements can be verified with a cost-effective system to check whether the final software meets those requirements. The requirements are verified with the help of reviews.
8. Traceability: The SRS is traceable if the origin of each of the requirements is clear and if it facilitates the referencing of each condition in future development or enhancement documentation.
9. Design Independence: There should be an option to select from multiple design alternatives for the final system. More specifically, the SRS should not contain any implementation details.
10. Testability: An SRS should be written in such a method that it is simple to generate test cases and test plans from the report.
11. Understandable by the customer: An end user may be an expert in his/her explicit domain but might not be trained in computer science. Hence, the purpose of formal notations and symbols should be avoided too as much extent as possible. The language should be kept simple and clear.
12. The right level of abstraction: If the SRS is written for the requirements stage, the details should be explained explicitly. Whereas, for a feasibility study, fewer analysis can be used. Hence, the level of abstraction modifies according to the objective of the SRS.

Object-oriented design using the UML
Unified Modeling Language (UML) Diagrams

Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed. It is quite similar to blueprints used in other fields of engineering. UML is not a programming language, it is rather a visual language.
Understanding and effectively using UML can significantly improve the quality and clarity of your software designs. Our specialized course on System design provides detailed guidance and practical examples to help you master this visual language. By integrating UML into your workflow, you can create more comprehensive and communicative system models.

Types of UML Diagrams



UML Diagrams

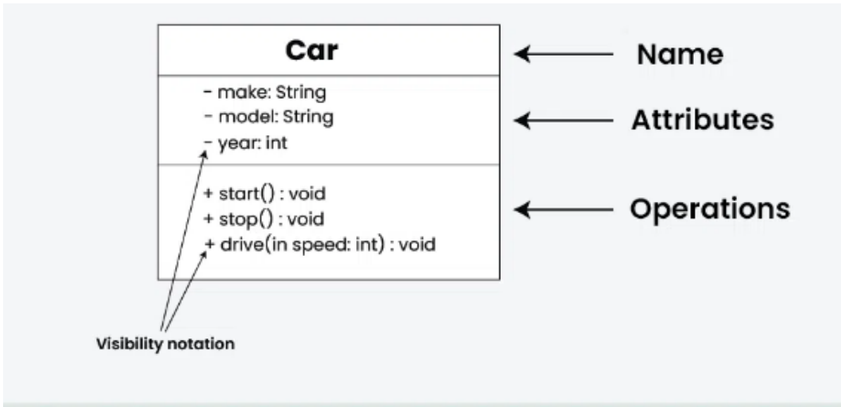
Class Diagram

What are class Diagrams?

Class diagrams are a type of UML (Unified Modeling Language) diagram used in software engineering to visually represent the structure and relationships of classes within a system i.e. used to construct and visualize object-oriented systems.

In these diagrams, classes are depicted as boxes, each containing three compartments for the class name, attributes, and methods. Lines connecting classes illustrate associations, showing relationships such as one-to-one or one-to-many.

Class diagrams provide a high-level overview of a system's design, helping to communicate and document the structure of the software. They are a fundamental tool in object-oriented design and play a crucial role in the software development lifecycle.

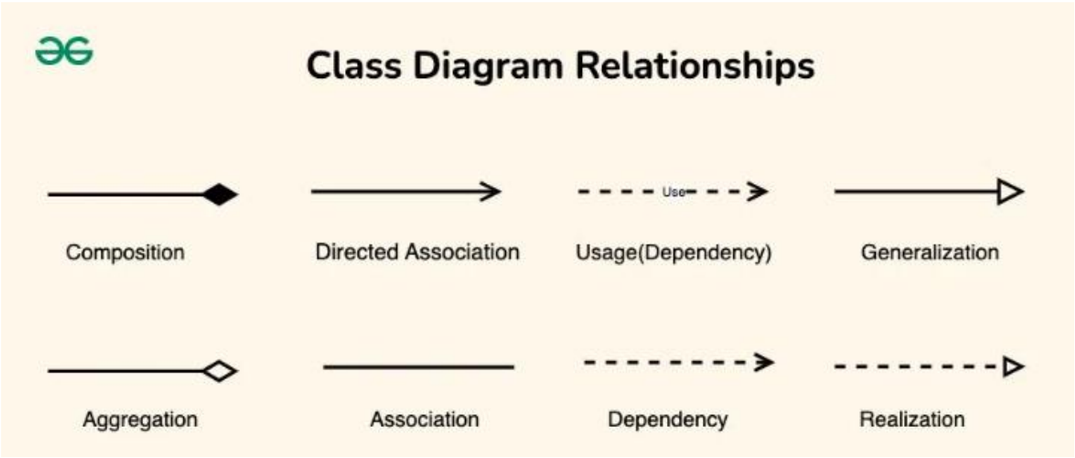


Class Notation

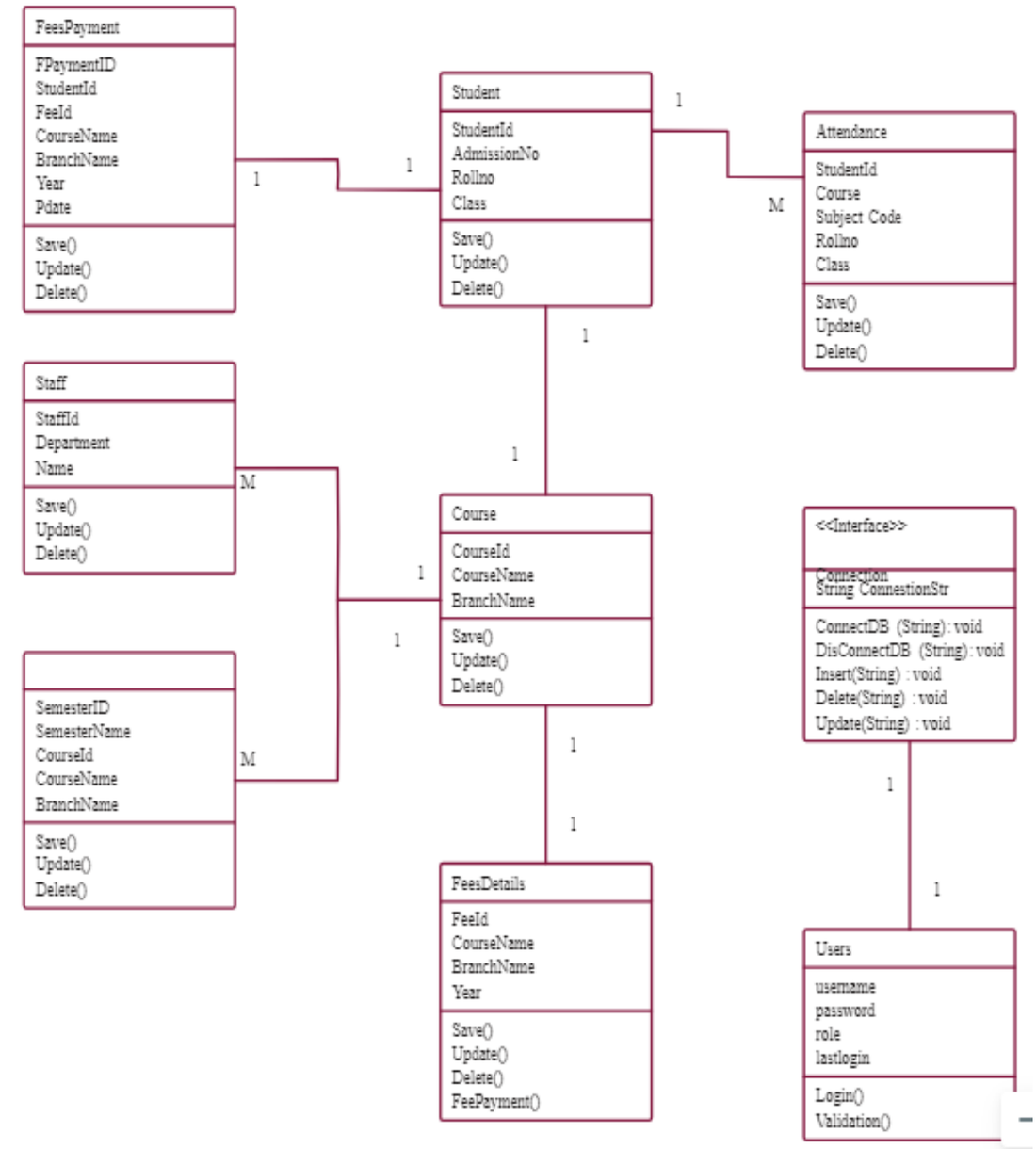
1. **Class Name:**
   - The name of the class is typically written in the top compartment of the class box and is centered and bold.
2. **Attributes:**
   - Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.
3. **Methods:**
   - Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.
4. **Visibility Notation:**
   - Visibility notations indicate the access level of attributes and methods. Common visibility notations include:
     - + for public (visible to all classes)
     - - for private (visible only within the class)
     - # for protected (visible to subclasses)
     - ~ for package or default visibility (visible to classes in the same package)

Relationships between classes

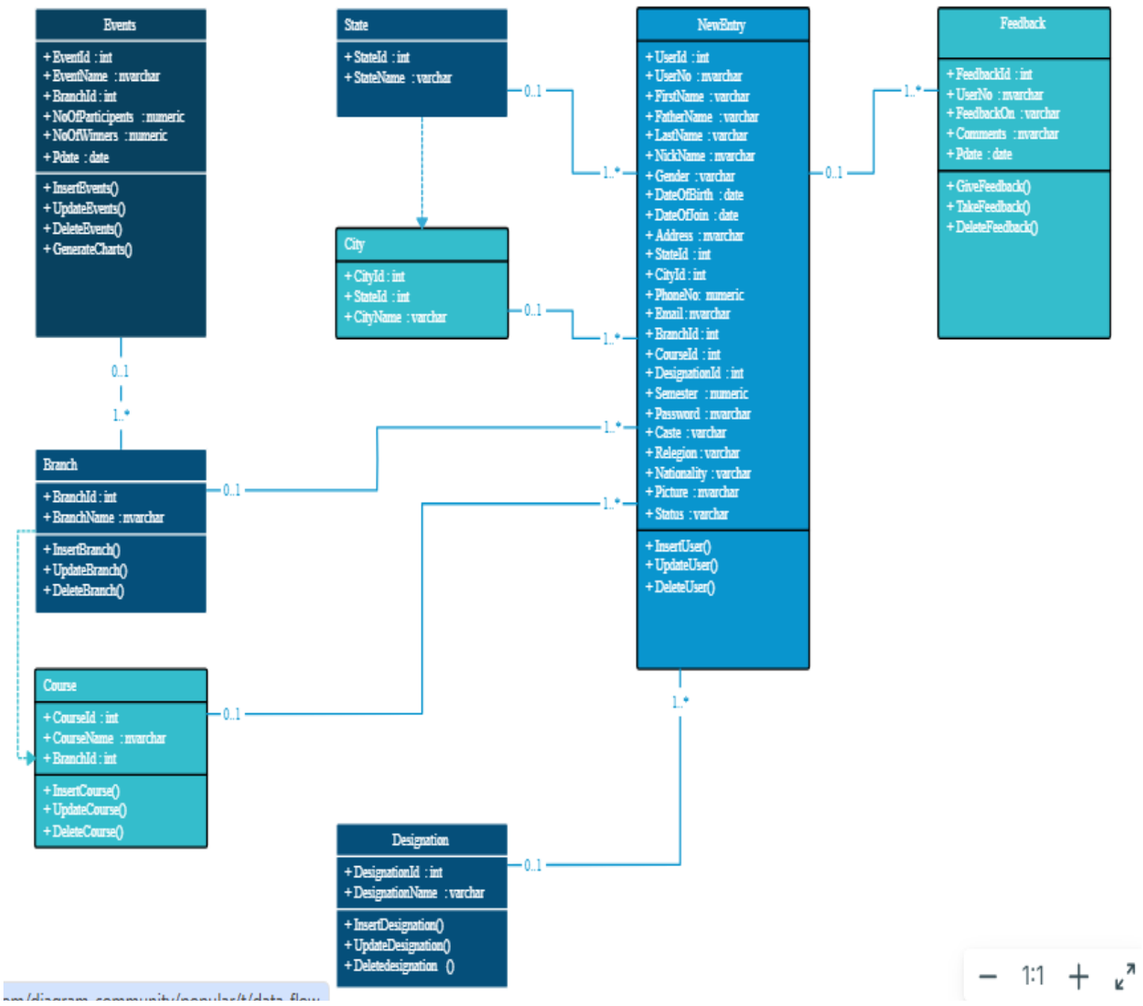
In class diagrams, relationships between classes describe how classes are connected or interact with each other within a system. There are several types of relationships in object-oriented modeling, each serving a specific purpose. Here are some common types of relationships in class diagrams:



College Management System - Class Diagram



CMS Class Diagram

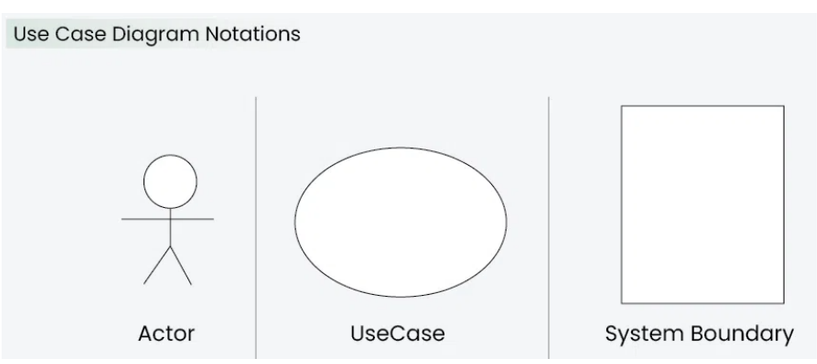


Use Case Diagram

A Use Case Diagram in Unified Modeling Language (UML) is a visual representation that illustrates the interactions between users (actors) and a system. It captures the functional requirements of a system, showing how different users engage with various use cases, or specific functionalities, within the system. Use case diagrams provide a high-level overview of a system's behavior, making them useful for stakeholders, developers, and analysts to understand how a system is intended to operate from the user's perspective, and how different processes relate to one another.

What is a Use Case Diagram in UML?

A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals. It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.



1. Actors

Actors are external entities that interact with the system. These can include users, other systems, or hardware devices. In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes. Proper identification and understanding of actors are crucial for accurately modeling system behavior.

2. Use Cases

Use cases are like scenes in the play. They represent specific things your system can do. In the online shopping system, examples of use cases could be “Place Order,” “Track Delivery,” or “Update Product Information”. Use cases are represented by ovals.

3. System Boundary

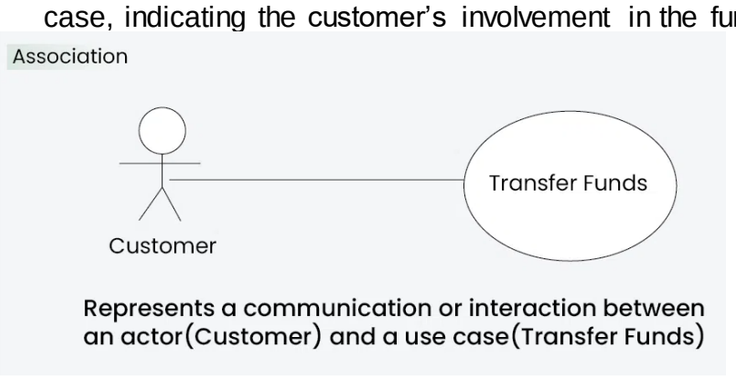
The system boundary is a visual representation of the scope or limits of the system you are modeling. It defines what is inside the system and what is outside. The boundary helps to establish a clear distinction between the elements that are part of the system and those that are external to it. The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system.

Association Relationship

The Association Relationship represents a communication or interaction between an actor and a use case. It is depicted by a line connecting the actor to the use case. This relationship signifies that the actor is involved in the functionality described by the use case.

Example: Online Banking System

- **Actor:** Customer
- **Use Case:** Transfer Funds
- **Association:** A line connecting the “Customer” actor to the “Transfer Funds” use case, indicating the customer’s involvement in the funds transfer process.

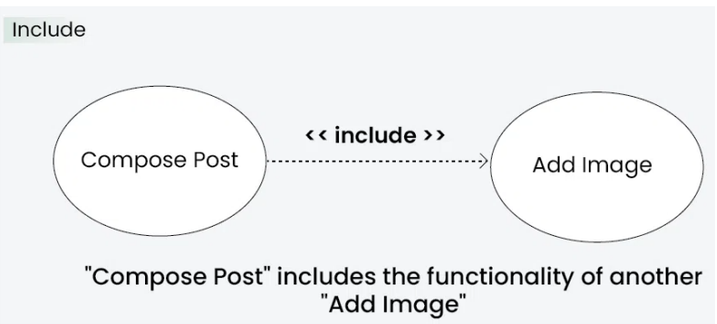


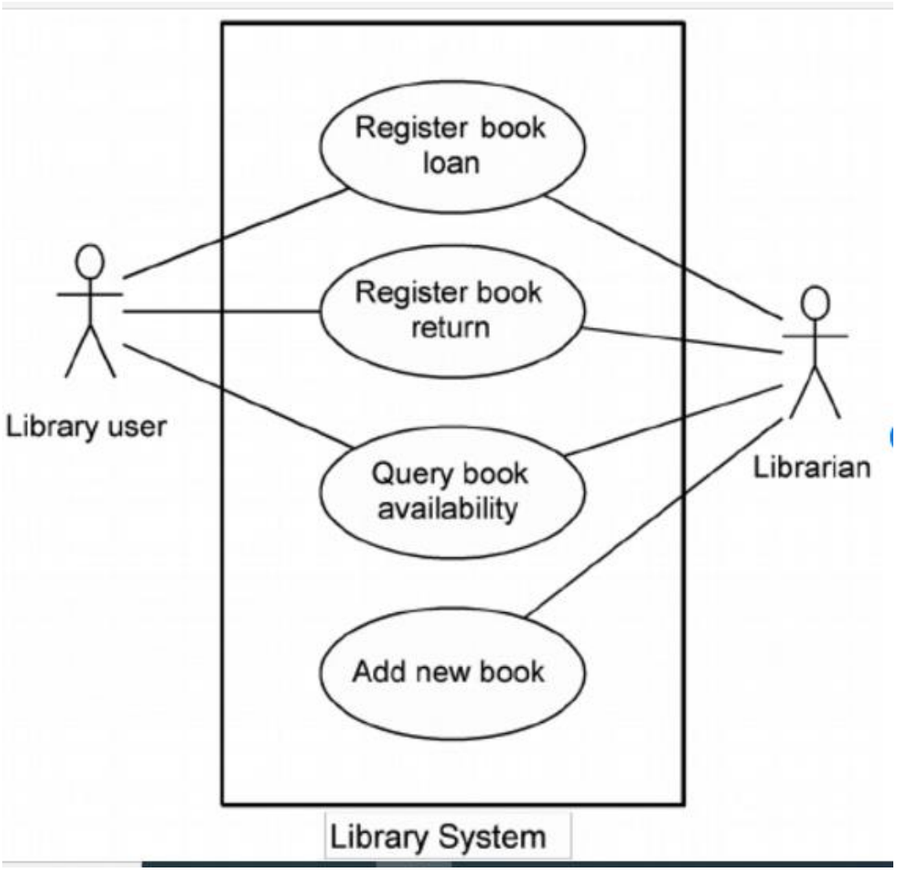
Include Relationship

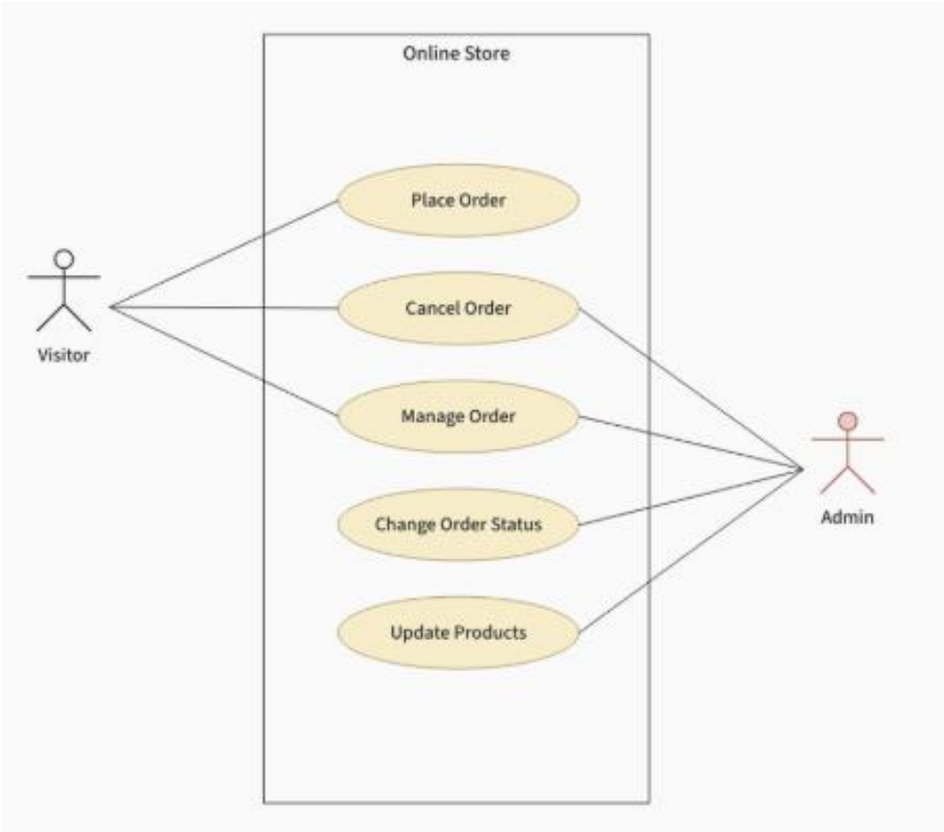
The Include Relationship indicates that a use case includes the functionality of another use case. It is denoted by a dashed arrow pointing from the including use case to the included use case. This relationship promotes modular and reusable design.

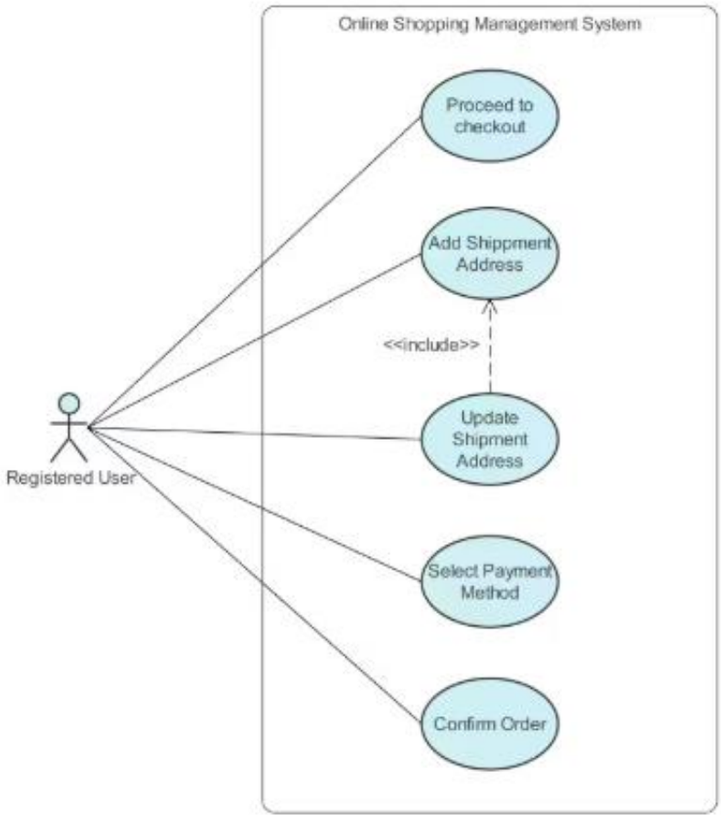
Example: Social Media Posting

- **Use Cases:** Compose Post, Add Image
- **Include Relationship:** The “Compose Post” use case includes the functionality of “Add Image.” Therefore, composing a post includes the action of adding an image.




Register book loan
Register book return
Query book availability
Add new book
Library user
Librarian
Library System


Online Store
Place Order
Cancel Order
Manage Order
Change Order Status
Update Products
Visitor
Admin



Sequence diagram

What are Sequence Diagrams?

Sequence diagrams are a type of UML (Unified Modeling Language) diagram that visually represent the interactions between objects or components in a system over time. They focus on the order and timing of messages or events exchanged between different system elements. The diagram captures how objects communicate with each other through a series of messages, providing a clear view of the sequence of operations or processes.

Why use Sequence Diagrams?

Sequence diagrams are used because they offer a clear and detailed visualization of the interactions between objects or components in a system, focusing on the order and timing of these interactions. Here are some key reasons for using sequence diagrams:

- **Visualizing Dynamic Behaviour**: Sequence diagrams depict how objects or systems interact with each other in a sequential manner, making it easier to understand dynamic processes and workflows.
- **Clear Communication**: They provide an intuitive way to convey system behavior, helping teams understand complex interactions without diving into code.
- **Use Case Analysis**: Sequence diagrams are useful for analyzing and representing use cases, making it clear how specific processes are executed within a system.
- **Designing System Architecture**: They assist in defining how various components or services in a system communicate, which is essential for designing complex, distributed systems or service-oriented architectures.
- **Documenting System Behaviour**: Sequence diagrams provide an effective way to document how different parts of a system work together, which can be useful for both developers and maintenance teams.
- **Debugging and Troubleshooting**: By modeling the sequence of interactions, they help identify potential bottlenecks, inefficiencies, or errors in system processes.

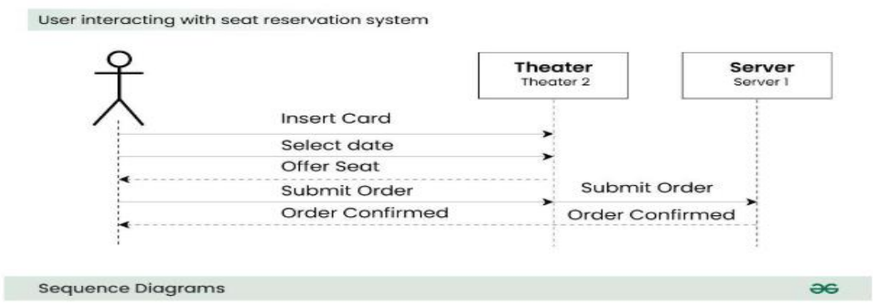
Sequence Diagram Notations

1. Actors

An actor in a UML diagram represents a type of role where it interacts with the system and its objects. It is important to note here that an actor is always outside the scope of the system we aim to model using the UML diagram.

For example:

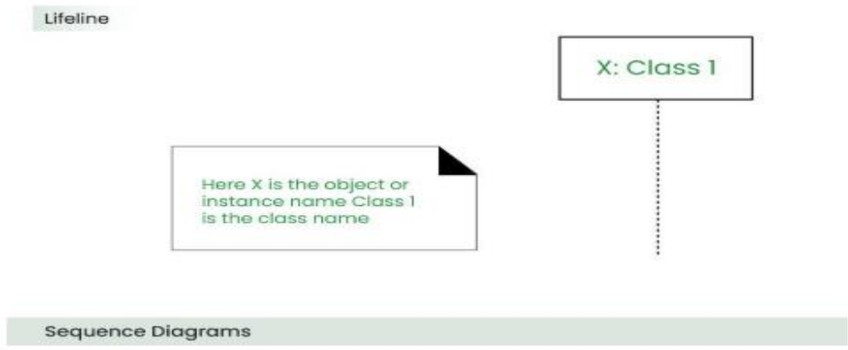
Here the user in seat reservation system is shown as an actor where it exists outside the system and is not a part of the system.



2. Lifelines

A lifeline is a named element which depicts an individual participant in a sequence diagram. So basically each instance in a sequence diagram is represented by a lifeline. Lifeline elements are located at the top in a sequence diagram. The standard in UML for naming a lifeline follows the following format:

Instance Name : Class Name

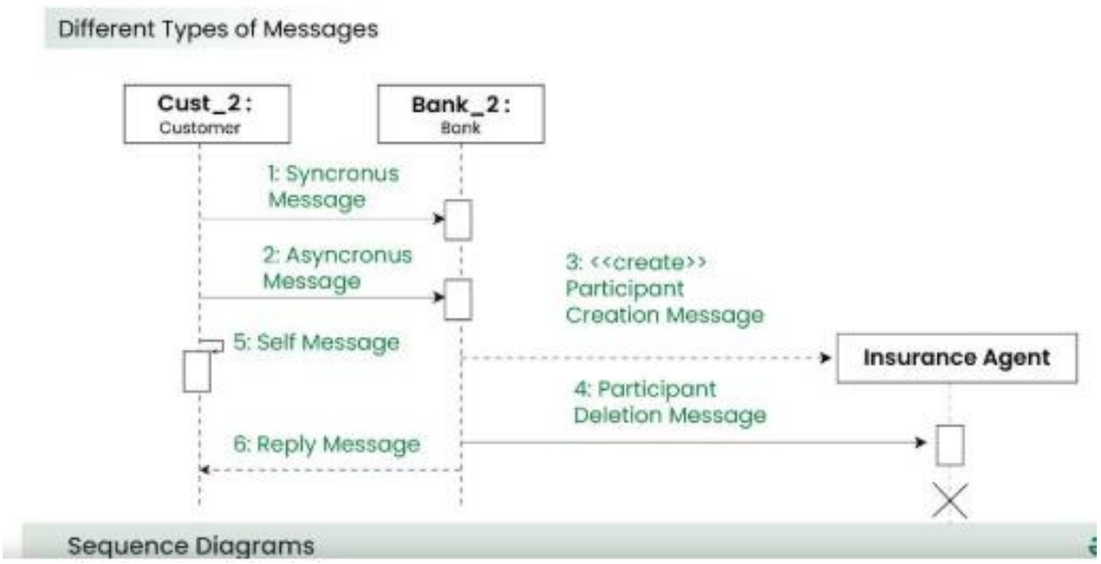


We display a lifeline in a rectangle called head with its name and type. The head is located on top of a vertical dashed line (referred to as the stem) as shown above.

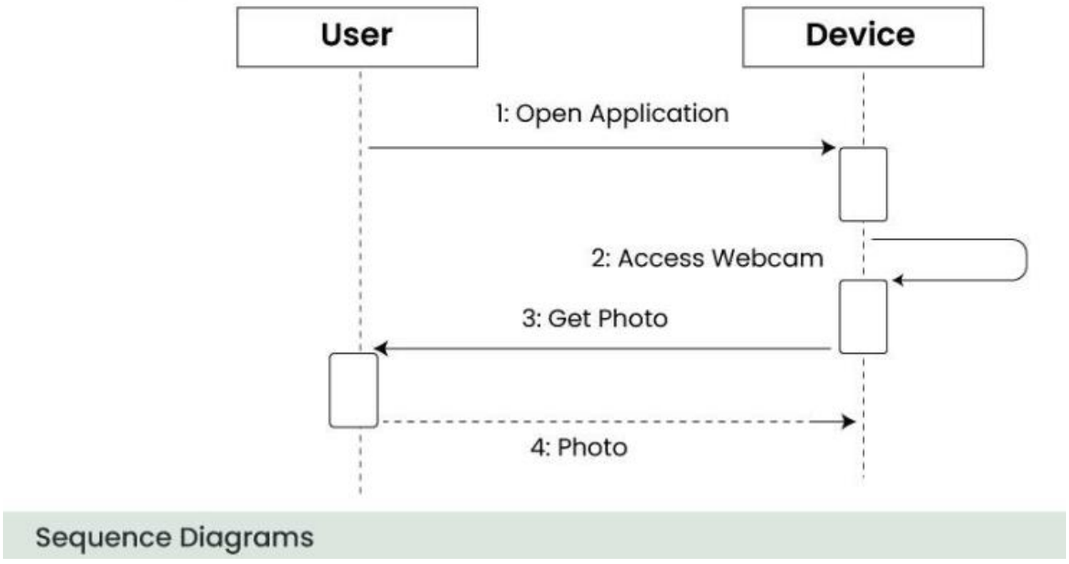
3. Messages

Communication between objects is depicted using messages. The messages appear in a sequential order on the lifeline.

- We represent messages using arrows.
- Lifelines and messages form the core of a sequence diagram.

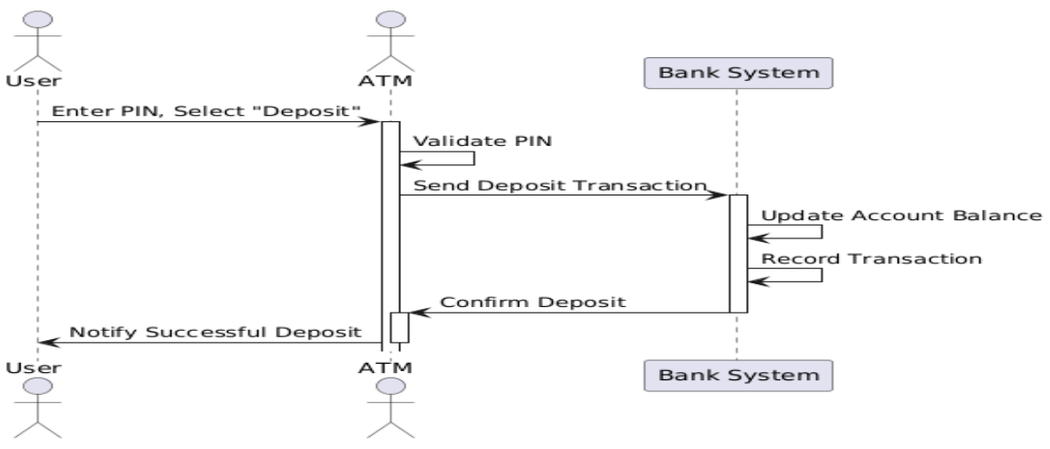


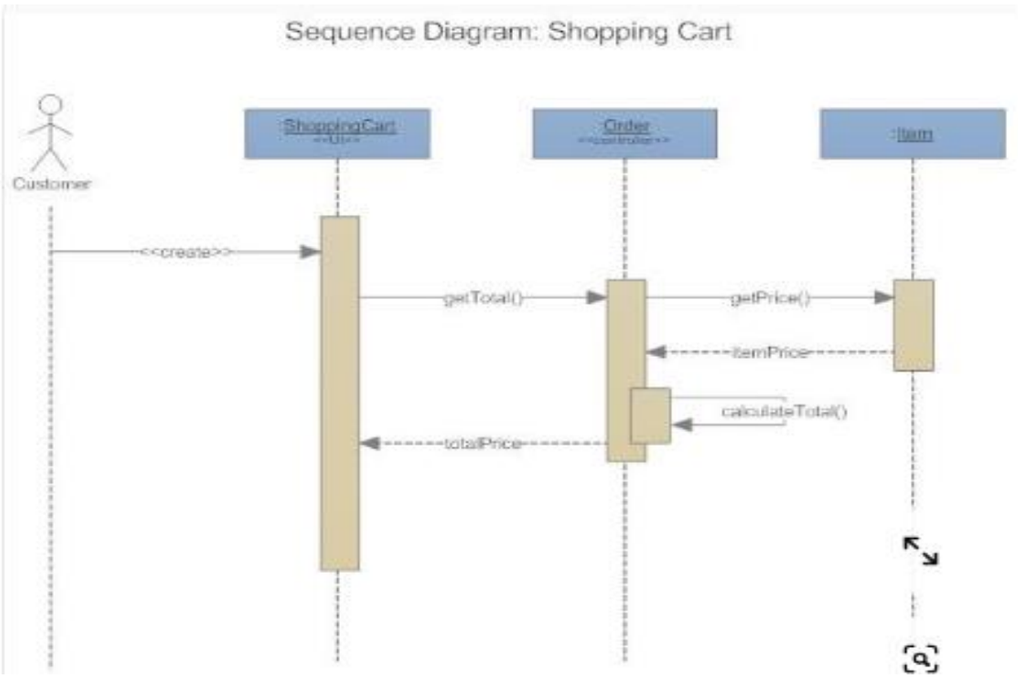
Consider the scenario where the device requests a photo from the user.



Case Study: A Simple Banking Transaction

Let's consider a simple banking transaction scenario: a user deposits money in ATM.





State Machine Diagrams

Statechart diagram describes the flow of control from one state to another state. States are defined as a condition in which an object exists and it changes when some event is triggered. The most important purpose of Statechart diagram is to model lifetime of an object from creation to termination.

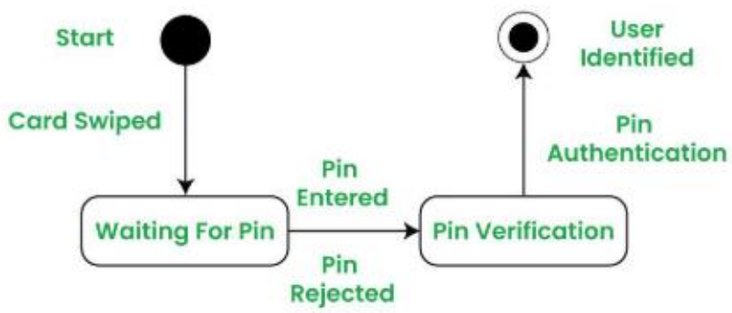
Following are the main purposes of using Statechart diagrams −

- To model the dynamic aspect of a system.
- To model the life time of a reactive system.
- To describe different states of an object during its life time.
- Define a state machine to model the states of an object.

Let's understand State Machine Diagram with the help user verification example:

Example:

A State Machine Diagram for user verification



State Machine Diagrams | Unified Modeling Language (UML)

Basic components and notations of a State Machine diagram

Initial state

We use a black filled circle represent the initial state of a System or a Class.

Transition

We use a solid arrow to represent the transition or change of control from one state to another. The arrow is labelled with the event which causes the change in state.

State

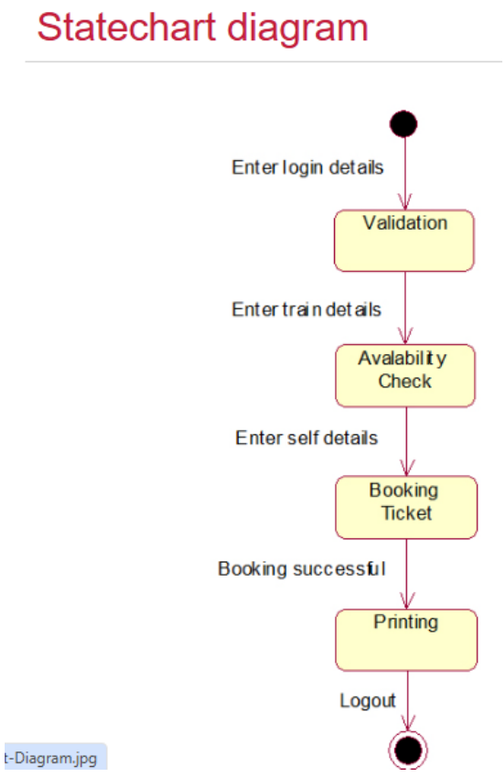
We use a rounded rectangle to represent a state. A state represents the conditions or circumstances of an object of a class at an instant of time.

Composite state

We use a rounded rectangle to represent a composite state also. We represent a state with internal activities using a composite state.

Final State

We use a filled circle within a circle notation to represent the final state in a state machine diagram.



Activity Diagrams

What is an Activity Diagram?

Activity diagrams show the steps involved in how a system works, helping us understand the flow of control. They display the order in which activities happen and whether they occur one after the other (sequential) or at the same time (concurrent). These diagrams help explain what triggers certain actions or events in a system.

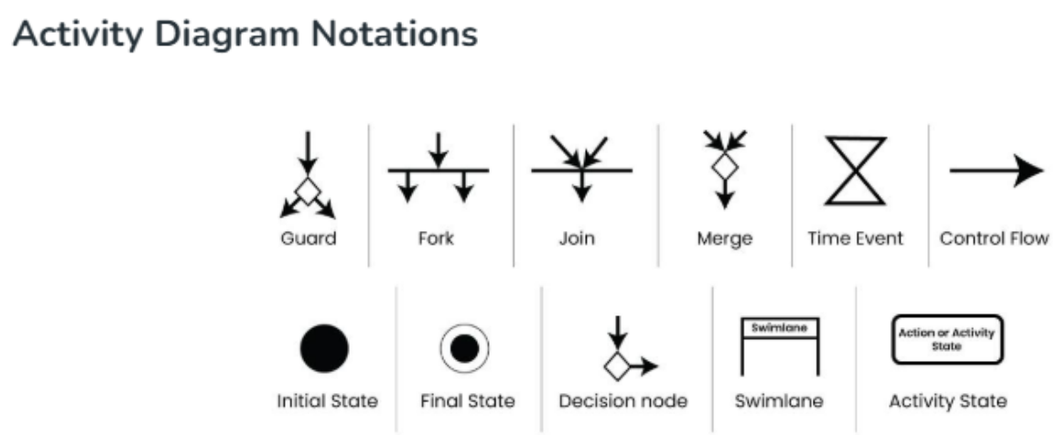
- An activity diagram starts from an initial point and ends at a final point, showing different decision paths along the way.
- They are often used in business and process modeling to show how a system behaves over time.

When to use Activity Diagram?

Activity diagrams are useful in several scenarios, especially when you need to visually represent the flow of processes or behaviors in a system. Here are key situations when you should use an activity diagram:

1. **Modelling Workflows or Processes**: When you need to map out a business process, workflow, or the steps involved in a use case, activity diagrams help visualize the flow of activities.

2. **Concurrent or Parallel Processing**: If your system or process involves activities happening simultaneously, an activity diagram can clearly show the parallel flow of tasks.
3. **Understanding the Dynamic Behaviour**: When it's essential to depict how a system changes over time and moves between different states based on events or conditions, activity diagrams are effective.
4. **Clarifying Complex Logic**: Use an activity diagram to simplify complex decision-making processes with branching paths and different outcomes.
5. **System Design and Analysis**: During the design phase of a software system, activity diagrams help developers and stakeholders understand how different parts of the system interact dynamically.
6. **Describing Use Cases**: They are useful for illustrating the flow of control within a use case, showing how various components of the system interact during its execution.



1. Initial State

The starting state before an activity takes place is depicted using the initial state.

2. Action or Activity State

An activity represents execution of an action on objects or by objects. We represent an activity using a rectangle with rounded corners. Basically any action or event that takes place is represented using an activity.

3. Action Flow or Control flows

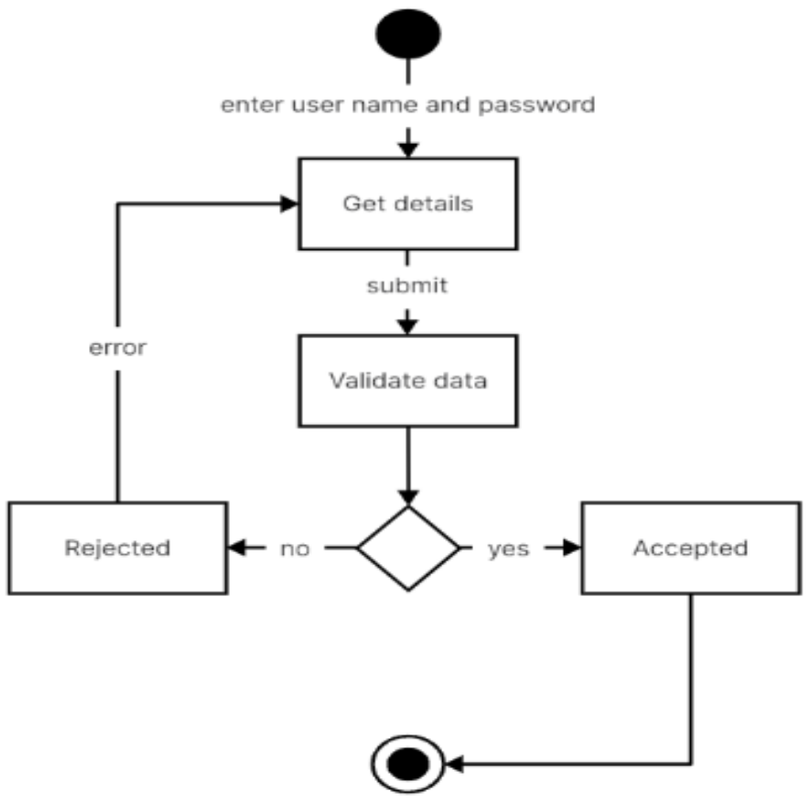
Action flows or Control flows are also referred to as paths and edges. They are used to show the transition from one activity state to another activity state.

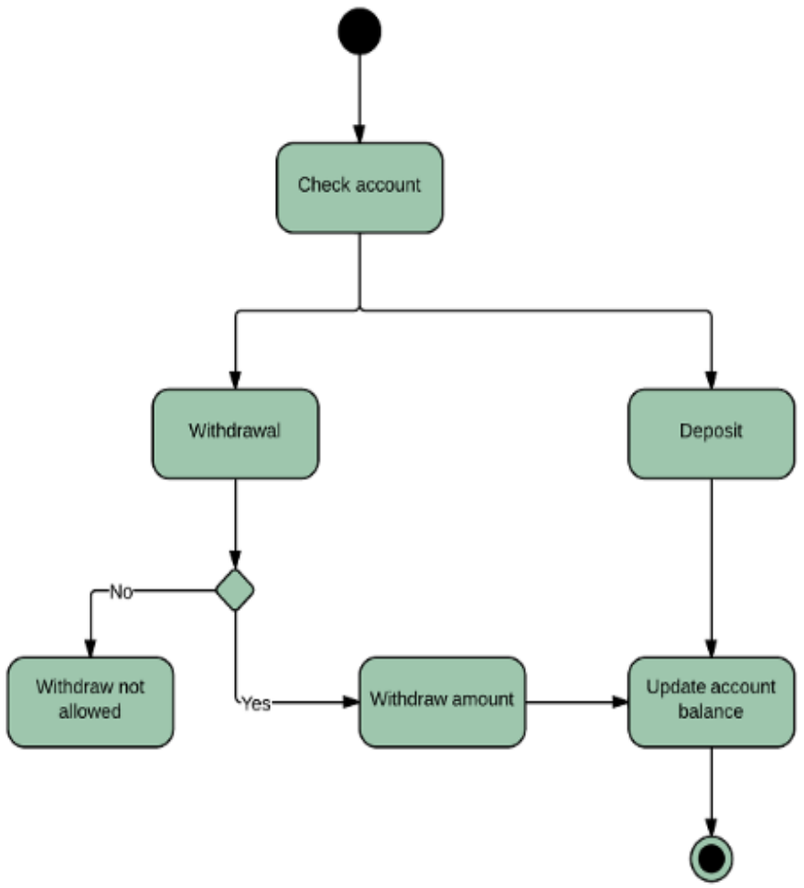
4. Decision node and Branching

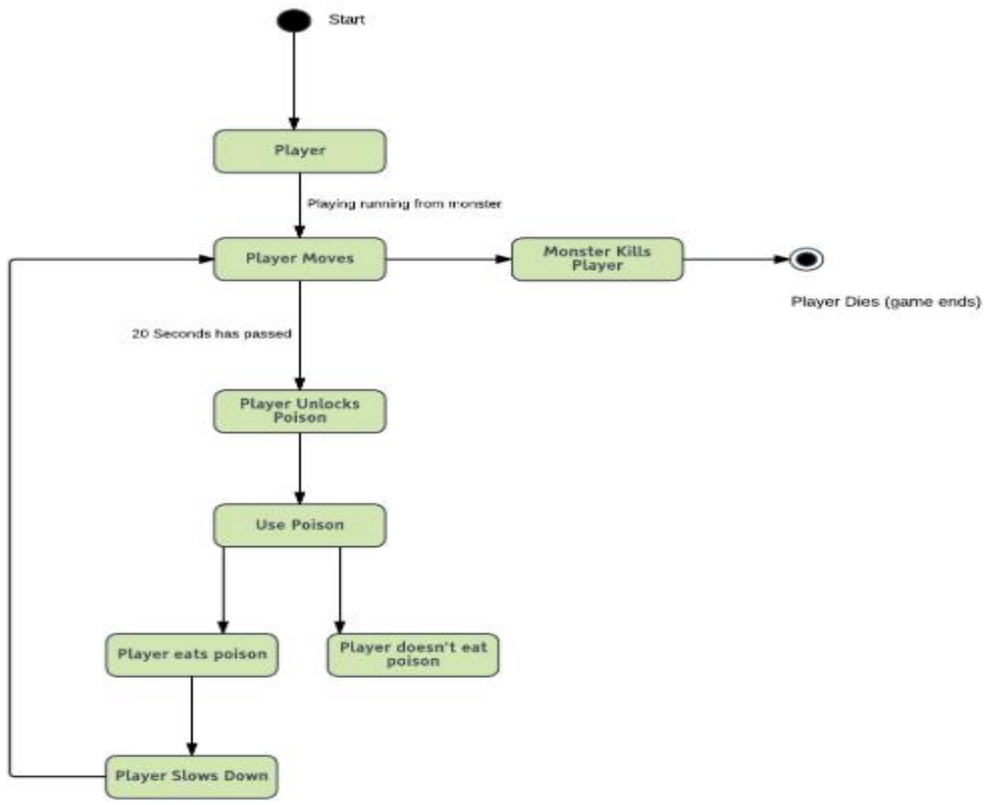
When we need to make a decision before deciding the flow of control, we use the decision node. The outgoing arrows from the decision node can be labelled with conditions or guard expressions. It always includes two or more output arrows.

5. Guard

A Guard refers to a statement written next to a decision node on an arrow sometimes within square brackets.


enter user name and password
Get details
submit
error
Validate data
Rejected
no
yes
Accepted


Check account
Withdrawal
Deposit
No
Withdraw not allowed
Yes
Withdraw amount
Update account balance


Start
Player
Playing running from monster
Player Moves
Monster Kills Player
Player Dies (game ends)
20 Seconds has passed
Player Unlocks Poison
Use Poison
Player eats poison
Player doesn't eat poison
Player Slows Down

UNIT II

Architectural Design

The software needs an architectural design to represent the design of the software. IEEE defines architectural design as "the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system." The software that is built for computer-based systems can exhibit one of these many architectural styles.

System Category Consists of

- A set of components (eg: a database, computational modules) that will perform a function required by the system.
- The set of connectors will help in coordination, communication, and cooperation between the components.
- Conditions that defines how components can be integrated to form the system.
- Semantic models that help the designer to understand the overall properties of the system.

The use of architectural styles is to establish a structure for all the components of the system.

Taxonomy of Architectural Styles

1] Data centered architectures:

- A data store will reside at the center of this architecture and is accessed frequently by the other components that update, add, delete, or modify the data present within the store.
- The figure illustrates a typical data-centered style. The client software accesses a central repository. Variations of this approach are used to transform the repository into a blackboard when data related to the client or data of interest for the client change the notifications to client software.
- This data-centered architecture will promote integrability. This means that the existing components can be changed and new client components can be added to the architecture without the permission or concern of other clients.
- Data can be passed among clients using the blackboard mechanism.

Advantages of Data centered architecture:

- Repository of data is independent of clients
- Client work independent of each other
- It may be simple to add additional clients.
- Modification can be very easy

Data centered architecture

2] Data flow architectures:

- This kind of architecture is used when input data is transformed into output data through a series of computational manipulative components.
- The figure represents pipe-and-filter architecture since it uses both pipe and filter and it has a set of components called filters connected by lines.
- Pipes are used to transmitting data from one component to the next.

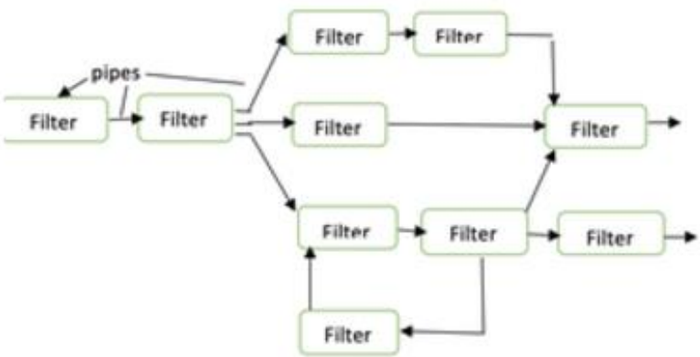
- Each filter will work independently and is designed to take data input of a certain form and produces data output to the next filter of a specified form. The filters don't require any knowledge of the working of neighboring filters.
- If the data flow degenerates into a single line of transforms, then it is termed as batch sequential. This structure accepts the batch of data and then applies a series of sequential components to transform it.

Advantages of Data Flow architecture:

- It encourages upkeep, repurposing, and modification.
- With this design, concurrent execution is supported.

Disadvantage of Data Flow architecture:

- It frequently degenerates to batch sequential system
- Data flow architecture does not allow applications that require greater user engagement.
- It is not easy to coordinate two different but related streams

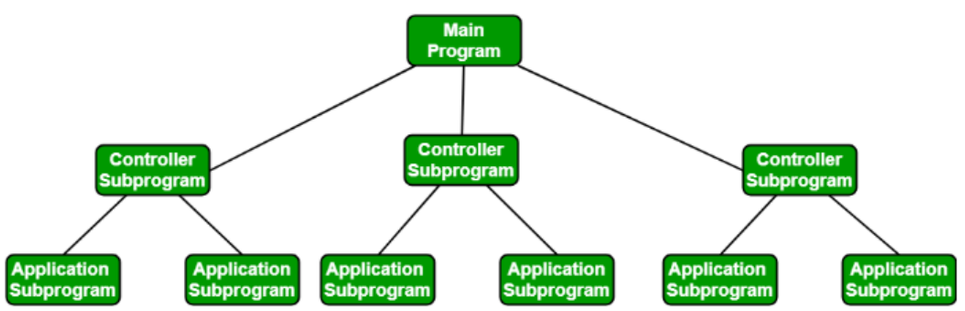


Data Flow architecture

3] Call and Return architectures

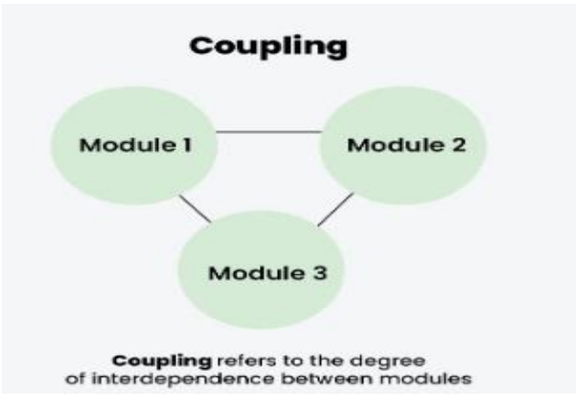
It is used to create a program that is easy to scale and modify. Many sub-styles exist within this category. Two of them are explained below.

- **Remote procedure call architecture:** This components is used to present in a main program or sub program architecture distributed among multiple computers on a network.
- **Main program or Subprogram architectures:** The main program structure decomposes into number of subprograms or function into a control hierarchy. Main program contains number of subprograms that can invoke other components.

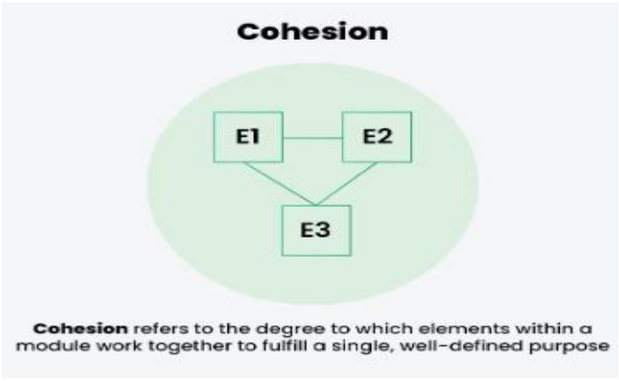


What is Coupling and Cohesion?

Coupling refers to the degree of interdependence between software modules. High coupling means that modules are closely connected and changes in one module may affect other modules. Low coupling means that modules are independent, and changes in one module have little impact on other modules.
If you're interested in learning how to implement coupling and cohesion effectively in your projects, the System Design Course delves deeper into these principles, helping you craft more maintainable and scalable code structures.



Cohesion refers to the degree to which elements within a module work together to fulfil a single, well-defined purpose. High cohesion means that elements are closely related and focused on a single purpose, while low cohesion means that elements are loosely related and serve multiple purposes.



Software Measurement:

A measurement is a manifestation of the size, quantity, amount, or dimension of a particular attribute of a product or process.

Software Measurement Principles

The software measurement process can be characterized by five activities-

1. **Formulation:** The derivation of software measures and metrics appropriate for the representation of the software that is being considered.
2. **Collection:** The mechanism used to accumulate data required to derive the formulated metrics.
3. **Analysis:** The computation of metrics and the application of mathematical tools.
4. **Interpretation:** The evaluation of metrics results in insight into the quality of the representation.
5. **Feedback:** Recommendation derived from the interpretation of product metrics transmitted to the software team.

Need for Software Measurement

Software is measured to:

- Create the quality of the current product or process.
- Anticipate future qualities of the product or process.
- Enhance the quality of a product or process.
- Regulate the state of the project concerning budget and schedule.
- Enable data-driven decision-making in project planning and control.
- Identify bottlenecks and areas for improvement to drive process improvement activities.
- Ensure that industry standards and regulations are followed.
- Give software products and processes a quantitative basis for evaluation.
- Enable the ongoing improvement of software development practices.

Software Metrics

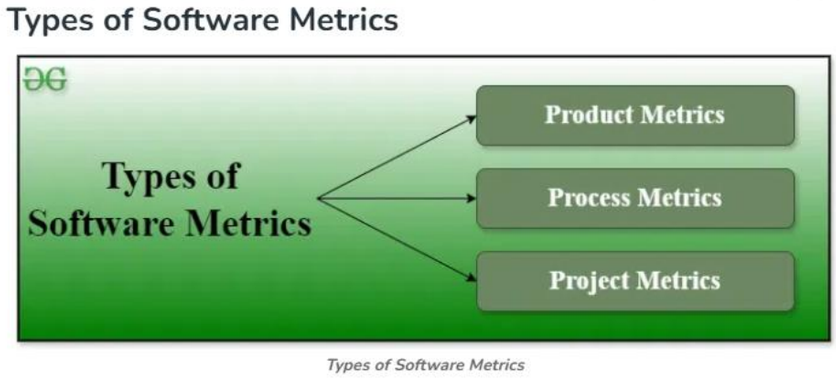
A metric is a measurement of the level at which any impute belongs to a system product or process.

Software metrics are a quantifiable or countable assessment of the attributes of a software product. There are 4 functions related to software metrics:

1. **Planning**
2. **Organizing**
3. **Controlling**
4. **Improving**

Characteristics of software Metrics

1. **Quantitative:** Metrics must possess a quantitative nature. It means metrics can be expressed in numerical values.
2. **Understandable:** Metric computation should be easily understood, and the method of computing metrics should be clearly defined.
3. **Applicability:** Metrics should be applicable in the initial phases of the development of the software.
4. **Repeatable:** When measured repeatedly, the metric values should be the same and consistent.
5. **Economical:** The computation of metrics should be economical.
6. **Language Independent:** Metrics should not depend on any programming language.



Types of Software Metrics

1. **Product Metrics:** Product metrics are used to evaluate the state of the product, tracing risks and undercover prospective problem areas. The ability of the team to control quality is evaluated. Examples include lines of code, cyclomatic complexity, code coverage, defect density, and code maintainability index.
2. **Process Metrics:** Process metrics pay particular attention to enhancing the long-term process of the team or organization. These metrics are used to optimize the development process and maintenance activities of software. Examples include effort variance, schedule variance, defect injection rate, and lead time.
3. **Project Metrics:** The project metrics describes the characteristic and execution of a project. Examples include effort estimation accuracy, schedule deviation, cost variance, and productivity. Usually measures-
   - Number of software developer
   - Staffing patterns over the life cycle of software
   - Cost and schedule
   - Productivity

Advantages of Software Metrics

1. Reduction in cost or budget.
2. It helps to identify the particular area for improvising.
3. It helps to increase the product quality.
4. Managing the workloads and teams.
5. Reduction in overall time to produce the product,.
6. It helps to determine the complexity of the code and to test the code with resources.
7. It helps in providing effective planning, controlling and managing of the entire product.

Disadvantages of Software Metrics

1. It is expensive and difficult to implement the metrics in some cases.
2. Performance of the entire team or an individual from the team can't be determined. Only the performance of the product is determined.
3. Sometimes the quality of the product is not met with the expectation.
4. It leads to measure the unwanted data which is wastage of time.
5. Measuring the incorrect data leads to make wrong decision making.

What is Project Size Estimation?

Project size estimation is determining the scope and resources required for the project.

1. It involves assessing the various aspects of the project to estimate the effort, time, cost, and resources needed to complete the project.
2. Accurate project size estimation is important for effective and efficient project planning, management, and execution.

Importance of Project Size Estimation

Here are some of the reasons why project size estimation is critical in project management:

1. **Financial Planning:** Project size estimation helps in planning the financial aspects of the project, thus helping to avoid financial shortfalls.
2. **Resource Planning:** It ensures the necessary resources are identified and allocated accordingly.
3. **Timeline Creation:** It facilitates the development of realistic timelines and milestones for the project.
4. **Identifying Risks:** It helps to identify potential risks associated with overall project execution.
5. **Detailed Planning:** It helps to create a detailed plan for the project execution, ensuring all the aspects of the project are considered.
6. **Planning Quality Assurance:** It helps in planning quality assurance activities and ensuring that the project outcomes meet the required standards.

Who Estimates Projects Size?

Here are the key roles involved in estimating the project size:

1. **Project Manager:** Project manager is responsible for overseeing the estimation process.
2. **Subject Matter Experts (SMEs):** SMEs provide detailed knowledge related to the specific areas of the project.
3. **Business Analysts:** Business Analysts help in understanding and documenting the project requirements.
4. **Technical Leads:** They estimate the technical aspects of the project such as system design, development, integration, and testing.
5. **Developers:** They will provide detailed estimates for the tasks they will handle.
6. **Financial Analysts:** They provide estimates related to the financial aspects of the project including labor costs, material costs, and other expenses.
7. **Risk Managers:** They assess the potential risks that could impact the projects' size and effort.
8. **Clients:** They provide input on project requirements, constraints, and expectations.

Lines of Code (LOC)

As the name suggests, LOC counts the total number of lines of source code in a project. The units of LOC are:

1. **KLOC:** Thousand lines of code
2. **NLOC:** Non-comment lines of code
3. **KDSI:** Thousands of delivered source instruction

- The size is estimated by comparing it with the existing systems of the same kind. The experts use it to predict the required size of various components of software and then add them to get the total size.
- It's tough to estimate LOC by analyzing the problem definition. Only after the whole code has been developed can accurate LOC be estimated. This statistic is of little utility to project managers because project planning must be completed before development activity can begin.
- Two separate source files having a similar number of lines may not require the same effort. A file with complicated logic would take longer to create than one with simple logic. Proper estimation may not be attainable based on LOC.
- The length of time it takes to solve an issue is measured in LOC. This statistic will differ greatly from one programmer to the next. A seasoned programmer can write the same logic in fewer lines than a newbie coder.

Advantages:

1. Universally accepted and is used in many models like COCOMO.
2. Estimation is closer to the developer's perspective.
3. Both people throughout the world utilize and accept it.
4. At project completion, LOC is easily quantified.
5. It has a specific connection to the result.
6. Simple to use.

Disadvantages:

1. Different programming languages contain a different number of lines.
2. No proper industry standard exists for this technique.
3. It is difficult to estimate the size using this technique in the early stages of the project.
4. When platforms and languages are different, LOC cannot be used to normalize.

Function Point Analysis

Count the number of functions of each proposed type:

Find the number of functions belonging to the following types:

- External Inputs: Functions related to data entering the system.
- External outputs: Functions related to data exiting the system.
- External Inquiries: They lead to data retrieval from the system but don't change the system.
- Internal Files: Logical files maintained within the system. Log files are not included here.
- External interface Files: These are logical files for other applications which are used by our system.

What is Project?

A project is a group of tasks that need to complete to reach a clear result. A project also defines as a set of inputs and outputs which are required to achieve a goal. Projects can vary from simple to difficult and can be operated by one person or a hundred

What is software project management?

Software Project Management (SPM) is a proper way of planning and leading software projects. It is a part of project management in which software projects are planned, implemented, monitored, and controlled.

Project Manager

A project manager is a character who has the overall responsibility for the planning, design, execution, monitoring, controlling and closure of a project. A project manager represents an essential role in the achievement of the projects.

A project manager is a character who is responsible for giving decisions, both large and small projects. The project manager is used to manage the risk and minimize uncertainty. Every decision the project manager makes must directly profit their project.

The role and responsibility of a software project manager

Software project managers may have to do any of the following tasks:

- **Planning:** The software project manager puts together the blueprint for the entire project. The project plan will define the scope, necessary resources, timeline, procedure for execution, communication strategy, and steps required for testing and maintenance.
- **Leading:** A software project manager assembles and leads the project team, which consists of developers, analysts, testers, graphic designers, and technical writers. Heading up a team requires excellent communication, people, and leadership skills.
- **Execution:** The person who manages software projects will supervise the successful execution of each stage of the project. This includes monitoring progress, conducting frequent team check-ins, and creating status reports.
- **Time management:** Staying on schedule is crucial to the successful completion of any project. This can be particularly challenging with the management of software projects because changes to the original plan are almost guaranteed as the project evolves. Software project managers must be experts in risk management and contingency planning to ensure progress in the face of roadblocks or changes.
- **Budget:** Like traditional project managers, professionals who manage software projects are tasked with creating a budget for a project and sticking to it as closely as possible, moderating spending and reallocating funds when necessary.
- **Maintenance:** Project management in software encourages constant product testing to discover and fix bugs early, adjust the end product to the customer's needs, and keep the project on target. The software project manager ensures the product is properly and consistently tested, evaluated, and adjusted accordingly.

Which Estimations Take Place During A Project?

There are six critical elements of a project that benefit from the use of project estimating techniques.

1. Cost

While managing software project, cost is one of the three primary constraints. The project will fail if you do not have sufficient funds to complete it. You can help set client expectations and ensure you have enough money to complete the work if you can accurately estimate project costs early on. Estimating software development costs entails determining how much money you'll need and when you'll need it.

2. Time

Another of the project's three main constraints is the lack of time. It is critical for project planning to be able to estimate both the overall project duration and the timing of individual tasks.

You can plan for people and resources to be available when you need them if you estimate your project schedule ahead of time. It also enables you to manage client expectations for key deliverables.

3. Size or Scope

The third major project constraint is scope. The project scope refers to all of the tasks that must be completed in order to complete the project or deliver a product. You can ensure that you have the right materials and expertise on the project by estimating how much work is involved and exactly what tasks must be completed. You need to know the scope and schedule to accurately estimate the budget.

4. Risk

Any unforeseen event that could positively or negatively impact your project is referred to as project risk. Estimating risk entails predicting what events will occur during the project's life cycle and how serious they will be.

You can better plan for potential issues and create risk management plans if you estimate what risks could affect your project and how they will affect it.

5. Resources

The assets you'll need to complete the project are known as project resources. Tools, people, materials, subcontractors, software, and other resources are all examples of resources. Resource management ensures that you have all of the resources you require and make the best use of them.

It's challenging to plan how you'll manage resources without knowing what you'll need and when. This can result in people sitting around doing nothing or materials arriving weeks after you need them.

6. Quality

Quality is concerned with the completion of project deliverables. Products that must adhere to stringent quality standards, such as environmental regulations, may require more money, time, and other resources than those with lower standards.

Estimating the level of quality required by the customer aids in the planning and estimating the remaining five aspects of your project. Because all six project factors are interconnected, forecasts for one can have an impact on forecasts for the other five.

What is COCOMO 2?

COCOMO 2 is another cost-estimating methodology utilized to calculate the cost of software development. It is a modified version of the original ***COCOMO*** that was developed at the ***University of Southern California***. It was mainly designed to resolve the shortcomings of the COCOMO 1 model. Its primary goal is to offer methodologies, quantitative analytic structure, and tools. It computes the total development time and effort that is based on the estimates of all individual subsystems.

It is based on the non-linear reuse formula. This approach assists in offering estimates that represent one standard deviation of the most common estimate. Five scale factors define its effort equation exponent, and it has 17 cost drivers attributed to it. It is useful in the software development cycle (SDLC) non-sequential, reuse, rapid development, and reengineering models.

COCOMO 2 estimation models

There are mainly 4 types of COCOMO 2 estimation models. These models are as follows:

1. Application Composition Model

It is intended for usage with reusable components to create prototype development estimates and operates on the basis of object points. It is more suited to prototype system development.

2. Early Design Model

After gathering requirements, the model is utilized during the system design phase. It generates estimates based on function points, which are subsequently converted into numerous lines of source code.

3. Reuse Model

This model computes the effort that is required to join reusable components and/or program code generated by design or program conversion tools. There are mainly two kinds of reused codes: white-box and black-box code. When there is no knowledge of the code, and no alteration is conducted in it, black box code is employed.

4. Post Architecture Model

A more accurate software estimate may be generated after designing the system architecture, and it is regarded as the most detailed of all models capable of producing an accurate estimate.

Generalized Activity Normalization Time Table (GANTT) chart

It is type of chart in which series of horizontal lines are present that show the amount of work done or production completed in given period of time in relation to amount planned for those projects.

It is simply used for graphical representation of schedule that helps to plan in an efficient way, coordinate, and track some particular tasks in project.

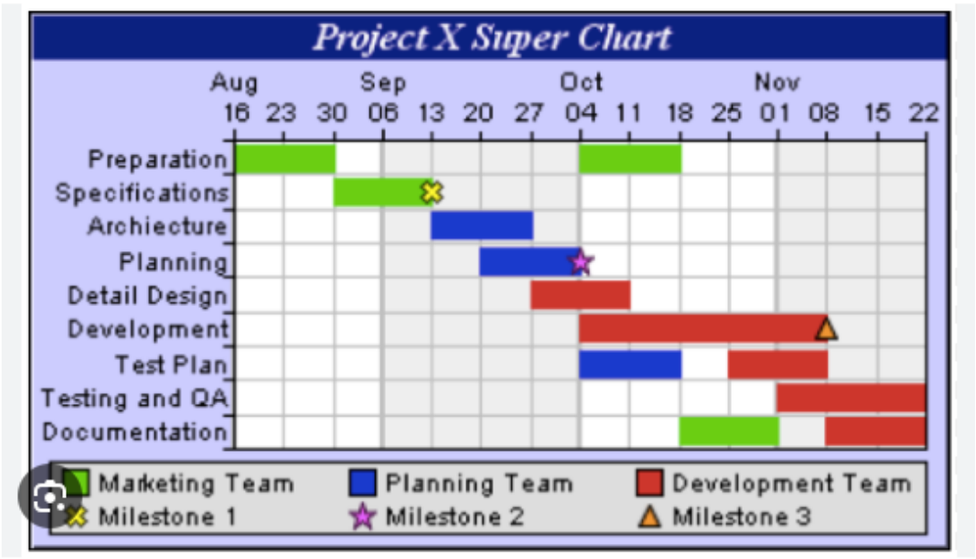
The purpose of Gantt chart is to emphasize scope of individual tasks. Hence set of tasks is given as input to Gantt chart. Gantt chart is also known as timeline chart. It can be developed for entire project or it can be developed for individual functions.

Gantt chart represents following things:

- All the tasks are listed at leftmost column.
- The horizontal bars indicate or represent required time by corresponding particular task.
- When occurring of multiple horizontal bars takes place at same time on calendar, then that means concurrency can be applied for performing particular tasks.
- The diamonds indicate milestones.

Advantages:

- **Simplify Project –** Gantt charts are generally used for simplifying complex projects.
- **Establish Schedule –** It simply establishes initial project schedule in which it mentions who is going to do what, when, and how much time it will take to complete it.
- **Provide Efficiency –** It brings efficiency in planning and allows team to better coordinate project activities.
- **Emphasize on scope –** It helps in emphasizing i.e., gives importance to scope of individual tasks.
- **Ease at understanding –** It makes it easy for stakeholders to understand timeline and brings clarity of dates.
- **Visualize project –** It helps in clearly visualizing project management, project tasks involved.
- **Organize thoughts and Highly visible –** It organizes your thoughts and can be highly visible so that everyone in enterprises can have basic level of understanding and have knowledge about what's happening in project even if they are not involved in working.
- **Make Practical and Realistic planning –** It makes the project planning practical and realistic as realistic planning generally helps to avoid any kind of delays and losses of many that can arise.



UNIT III

What is Risk Management?

Risk Management is a systematic process of recognizing, evaluating, and handling threats or risks that have an effect on the finances, capital, and overall operations of an organization. These risks can come from different areas, such as financial instability, legal issues, errors in strategic planning, accidents, and natural disasters.

The main goal of risk management is to predict possible risks and find solutions to deal with them successfully.

Risk management is important because it helps organizations to prepare for unexpected circumstances that can vary from small issues to major crises. By actively understanding, evaluating, and planning for potential risks, organizations can protect their financial health, continued operation, and overall survival.

Suppose In a software development project, one of the key developers unexpectedly falls ill and is unable to contribute to the product for an extended period.

Various Kinds of Risks in Software Development

1. **Schedule Risk :** Schedule related risks refers to time related risks or project delivery related planning risks. The wrong schedule affects the project development and delivery. These risks are mainly indicates to running behind time as a result project development doesn't progress timely and it directly impacts to delivery of project. Finally if schedule risks are not managed properly it gives rise to project failure and at last it affect to organization/company economy very badly. Some reasons for Schedule risks –
   - Time is not estimated perfectly
   - Improper resource allocation
   - Tracking of resources like system, skill, staff etc
   - Frequent project scope expansion
   - Failure in function identification and its' completion
2. **Budget Risk:** Budget related risks refers to the monetary risks mainly it occurs due to budget overruns. Always the financial aspect for the project should be managed as per decided but if financial aspect of project mismanaged then there budget concerns will arise by giving rise to budget risks. So proper finance distribution and management are required for the success of project otherwise it may lead to project failure. Some reasons for Budget risks –
   - Wrong/Improper budget estimation
   - Unexpected Project Scope expansion
   - Mismanagement in budget handling
   - Cost overruns
   - Improper tracking of Budget
3. **Operational Risks :** Operational risk refers to the procedural risks means these are the risks which happen in day-to-day operational activities during project development due to improper process implementation or some external operational risks. Some reasons for Operational risks –
   - Insufficient resources
   - Conflict between tasks and employees
   - Improper management of tasks
   - No proper planning about project
   - Less number of skilled people
   - Lack of communication and cooperation
   - Lack of clarity in roles and responsibilities
   - Insufficient training
4. **Technical Risks :** Technical risks refers to the functional risk or performance risk which means this technical risk mainly associated with functionality of product or performance part of the software product. Some reasons for Technical risks –
   - Frequent changes in requirement
   - Less use of future technologies
   - Less number of skilled employee
   - High complexity in implementation
   - Improper integration of modules

5. **Programmatic Risks :** Programmatic risks refers to the external risk or other unavoidable risks. These are the external risks which are unavoidable in nature. These risks come from outside and it is out of control of programs. Some reasons for Programmatic risks –
   - Rapid development of market
   - Running out of fund / Limited fund for project development
   - Changes in Government rules/policy
   - Loss of contracts due to any reason

More risks associated with software development

1. **Communication Risks:** Misunderstandings, mistakes and a general sense of confusion can result from inadequate or absent communication.
2. **Security Risks:** Vulnerabilities that might compromise the privacy, reliability or accessibility of the set are known as security risks and they have become common in a time.
3. **Quality Risks:** The risk associated with quality is the potential for a product to be delivered that does not meet end user satisfaction or required criteria.
4. **Risks associated with Law and Compliance:** Rules and laws are often overlooked when it comes to project development. Ignoring them may result in penalties, legal issues or just a lot of difficulties.
5. **Cost Risks:** Unexpected costs, changes in the project scope or excess funds may completely halt your financial plan.
6. **Market Risks:** The effectiveness of your programme in the market may be compromised by evolving technology trends, new competitors or shifting the customer wants.

The Risk management process

Risk management is a sequence of steps that help a software team to understand, analyze, and manage uncertainty. Risk management process consists of

- **Risks Identification.**
- **Risk Assessment.**
- **Risks Planning.**
- **Risk Monitoring**

Risk Identification

Risk identification refers to the systematic process of recognizing and evaluating potential threats or hazards that could negatively impact an organization, its operations, or its workforce. This involves identifying various types of risks, ranging from IT security threats like viruses and phishing attacks to unforeseen events such as equipment failures and extreme weather conditions.

Risk analysis

Risk analysis is the process of evaluating and understanding the potential impact and likelihood of identified risks on an organization. It helps determine how serious a risk is and how to best manage or mitigate it. Risk Analysis involves evaluating each risk's probability and potential consequences to prioritize and manage them effectively.

Risk Planning

Risk planning involves developing strategies and actions to manage and mitigate identified risks effectively. It outlines how to respond to potential risks, including

prevention, mitigation, and contingency measures, to protect the organization's objectives and assets.

Risk Monitoring

Risk monitoring involves continuously tracking and overseeing identified risks to assess their status, changes, and effectiveness of mitigation strategies. It ensures that risks are regularly reviewed and managed to maintain alignment with organizational objectives and adapt to new developments or challenges.

Risk Mitigation, Monitoring, and Management (RMMM) plan

A risk management technique is usually seen in the software Project plan. This can be divided into Risk Mitigation, Monitoring, and Management Plan (RMMM). In this plan, all works are done as part of risk analysis. As part of the overall project plan project manager generally uses this RMMM plan.

In some software teams, risk is documented with the help of a Risk Information Sheet (RIS). This RIS is controlled by using a database system for easier management of information i.e creation, priority ordering, searching, and other analysis. After documentation of RMMM and start of a project, risk mitigation and monitoring steps will start.

Risk Mitigation:

It is an activity used to avoid problems (Risk Avoidance).
Steps for mitigating the risks as follows.

1. Finding out the risk.
2. Removing causes that are the reason for risk creation.
3. Controlling the corresponding documents from time to time.
4. Conducting timely reviews to speed up the work.

Risk Monitoring:

It is an activity used for project tracking.
It has the following primary objectives as follows.

1. To check if predicted risks occur or not.
2. To ensure proper application of risk aversion steps defined for risk.
3. To collect data for future risk analysis.
4. To allocate what problems are caused by which risks throughout the project.

Risk Management and planning:

It assumes that the mitigation activity failed and the risk is a reality. This task is done by Project manager when risk becomes reality and causes severe problems. If the project manager effectively uses project mitigation to remove risks successfully then it is easier to manage the risks. This shows that the response that will be taken for each risk by a manager. The main objective of the risk management plan is the risk register. This risk register describes and focuses on the predicted threats to a software project.

Software Quality Assurance

Software Quality Assurance (SQA) is simply a way to assure quality in the software. It is the set of activities that ensure processes, procedures as well as standards are suitable for the project and implemented correctly.
Software Quality Assurance is a process that works parallel to Software Development. It focuses on improving the process of development of software so that problems can be prevented before they become major issues. Software Quality Assurance is a kind of Umbrella activity that is applied throughout the software process.

What is quality?

Quality in a product or service can be defined by several measurable characteristics. Each of these characteristics plays a crucial role in determining the overall quality.

Elements of Software Quality Assurance (SQA)

1. **Standards:** The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.
2. **Reviews and audits:** Technical reviews are a quality control activity performed by software engineers for software engineers. Their intent is to uncover errors. Audits are a type of review performed by SQA personnel (people employed in an organization) with the intent of ensuring that quality guidelines are being followed for software engineering work.
3. **Testing:** Software testing is a quality control function that has one primary goal—to find errors. The job of SQA is to ensure that testing is properly planned and efficiently conducted for primary goal of software.
4. **Error/defect collection and analysis:** SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.
5. **Change management:** SQA ensures that adequate change management practices have been instituted.
6. **Education:** Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement which is key proponent and sponsor of educational programs.
7. **Security management:** SQA ensures that appropriate process and technology are used to achieve software security.
8. **Safety:** SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.
9. **Risk management**: The SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

Major Software Quality Assurance (SQA) Activities

1. **SQA Management Plan:** Make a plan for how you will carry out the SQA throughout the project. Think about which set of software engineering activities are the best for project. check level of SQA team skills.
2. **Set The Check Points:** SQA team should set checkpoints. Evaluate the performance of the project on the basis of collected data on different check points.
3. **Measure Change Impact:** The changes for making the correction of an error sometimes re introduces more errors keep the measure of impact of change on project. Reset the new change to check the compatibility of this fix with whole project.
4. **Multi testing Strategy:** Do not depend on a single testing approach. When you have a lot of testing approaches available use them.
5. **Manage Good Relations:** In the working environment managing good relations with other teams involved in the project development is mandatory. Bad relation of SQA team with programmers team will impact directly and badly on project. Don't play politics.
6. **Maintaining records and reports:** Comprehensively document and share all QA records, including test cases, defects, changes, and cycles, for stakeholder awareness and future reference.
7. **Reviews software engineering activities:** The SQA group identifies and documents the processes. The group also verifies the correctness of software product.
8. **Formalize deviation handling:** Track and document software deviations meticulously. Follow established procedures for handling variances.

Benefits of Software Quality Assurance (SQA)

1. SQA produces high quality software.
2. High quality application saves time and cost.
3. SQA is beneficial for better reliability.
4. SQA is beneficial in the condition of no maintenance for a long time.
5. High quality commercial software increase market share of company.
6. Improving the process of creating software.

7. Improves the quality of the software.
8. It cuts maintenance costs. Get the release right the first time, and your company can forget about it and move on to the next big thing. Release a product with chronic issues, and your business bogs down in a costly, time-consuming, never-ending cycle of repairs.

What is a Software Quality Assurance Plan?

A Quality Assurance Plan (QAP) is a document or set of documents that outlines the systematic processes, procedures, and standards for ensuring the quality of a product or service. It is a key component of quality management and is used in various industries to establish and maintain a consistent level of quality in deliverables. For a software product or service, an SQA plan will be used in conjunction with the typical development, prototyping, design, production, and release cycle. An SQA plan will include several components, such as purpose, references, configuration and management, tools, code controls, testing methodology, problem reporting and remedial measures, and more, for easy documentation and referencing.

Software Configuration Management

When we develop software, the product (software) undergoes many changes in their maintenance phase; we need to handle these changes effectively.

Several individuals (programs) works together to achieve these common goals. This individual produces several work product (SC Items) e.g., Intermediate version of modules or test data used during debugging, parts of the final product.

The elements that comprise all information produced as a part of the software process are collectively called a software configuration.As software development progresses, the number of Software Configuration elements (SCI's) grow rapidly.These are handled and controlled by SCM. This is where we require software configuration management.

SCM is the discipline which

- Identify change
- Monitor and control change
- Ensure the proper implementation of change made to the item.
- Auditing and reporting on the change made.

Configuration Management (CM) is a technic of identifying, organizing, and controlling modification to software being built by a programming team.

The objective is to maximize productivity by minimizing mistakes (errors).

CM is used to essential due to the inventory management, library management, and updation management of the items essential for the project.

Key objectives of SCM

1. **Control the evolution of software systems:** SCM helps to ensure that changes to a software system are properly planned, tested, and integrated into the final product.

2. **Enable collaboration and coordination:** SCM helps teams to collaborate and coordinate their work, ensuring that changes are properly integrated and that everyone is working from the same version of the software system.
3. **Provide version control:** SCM provides version control for software systems, enabling teams to manage and track different versions of the system and to revert to earlier versions if necessary.
4. **Facilitate replication and distribution:** SCM helps to ensure that software systems can be easily replicated and distributed to other environments, such as test, production, and customer sites.
5. SCM is a critical component of software development, and effective SCM practices can help to improve the quality and reliability of software systems, as well as increase efficiency and reduce the risk of errors.

The main Advantages of SCM

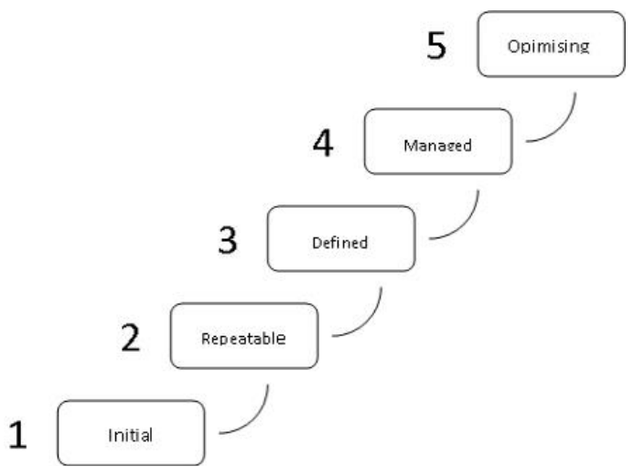
1. Improved productivity and efficiency by reducing the time and effort required to manage software changes.
2. Reduced risk of errors and defects by ensuring that all changes were properly tested and validated.
3. Increased collaboration and communication among team members by providing a central repository for software artifacts.
4. Improved quality and stability of software systems by ensuring that all changes are properly controlled and managed.

The main Disadvantages of SCM

1. Increased complexity and overhead, particularly in large software systems.
2. Difficulty in managing dependencies and ensuring that all changes are properly integrated.
3. Potential for conflicts and delays, particularly in large development teams with multiple contributors

The Capability Maturity Model (CMM)

The Capability Maturity Model (CMM) is a framework that lays out five maturity levels for continual process improvement. This framework is integral to most management systems that aim to improve the quality of development and delivery for all products and services.



The five Software Capability Maturity levels

1. Initial

The software process is characterised as ad hoc, and occasionally even chaotic. Few processes are defined, and success depends on individual effort.

2. Repeatable

Basic project management processes are established to track cost, schedule and functionality. The necessary process discipline is in place to repeat earlier successes on projects with similar applications.

3. Defined

The software process for both management and engineering activities is documented, standardised and integrated into all processes for the organisation. All projects use an

approved version of the organisation's standard software process for developing and maintaining software.

4. Managed

Detailed measures of the software process and product quality are collected. Both the software process and products are quantitatively understood and controlled.

5. Optimising

Continuous process improvement is enabled by quantitative feedback from the process and from piloting innovative ideas and technologies.

Software testing

Software testing is the process of evaluating and verifying that a software product or application does what it's supposed to do. The benefits of good testing include preventing bugs and improving performance.

Importance of Software Testing

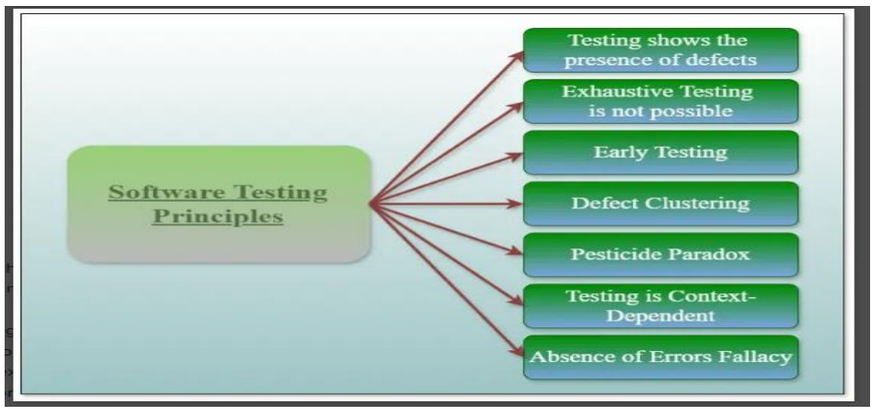
- **Defects can be identified early:** Software testing is important because if there are any bugs they can be identified early and can be fixed before the delivery of the software.
- **Improves quality of software:** Software Testing uncovers the defects in the software, and fixing them improves the quality of the software.
- **Increased customer satisfaction:** Software testing ensures reliability, security, and high performance which results in saving time, costs, and customer satisfaction.
- **Helps with scalability:** Software testing type non-functional testing helps to identify the scalability issues and the point where an application might stop working.
- **Saves time and money:** After the application is launched it will be very difficult to trace and resolve the issues, as performing this activity will incur more costs and time. Thus, it is better to conduct software testing at regular intervals during software development.

Differences between Verification and Validation

	Verification	**Validation**
Definition	Verification refers to the set of activities that ensure software correctly implements the specific function	Validation refers to the set of activities that ensure that the software that has been built is traceable to customer requirements.
Focus	It includes checking documents, designs, codes, and programs.	It includes testing and validating the actual product.
Type of Testing	Verification is the static testing.	Validation is dynamic testing.
Execution	It does *not* include the execution of the code.	It includes the execution of the code.
Methods Used	Methods used in verification are reviews, walkthroughs, inspections and desk-checking.	Methods used in validation are Black Box Testing, White Box Testing and non-functional testing.

	Verification	Validation
Purpose	It checks whether the software conforms to specifications or not.	It checks whether the software meets the requirements and expectations of a customer or not.
Bug	It can find the bugs in the early stage of the development.	It can only find the bugs that could not be found by the verification process.
Goal	The goal of verification is application and software architecture and specification.	The goal of validation is an actual product.
Responsibility	Quality assurance team does verification.	Validation is executed on software code with the help of testing team.
Timing	It comes before validation.	It comes after verification.
Human or Computer	It consists of checking of documents/files and is performed by human.	It consists of execution of program and is performed by computer.
Lifecycle	After a valid and complete specification the verification starts.	Validation begins as soon as project starts.
Error Focus	Verification is for prevention of errors.	Validation is for detection of errors.
Another Terminology	Verification is also termed as white box testing or static testing as work product goes through reviews.	Validation can be termed as black box testing or dynamic testing as work product is executed.
Performance	Verification finds about 50 to 60% of the defects.	Validation finds about 20 to 30% of the defects.
Stability	Verification is based on the opinion of reviewer and may change from person to person.	Validation is based on the fact and is often stable.

Principles of software testing



1. Testing shows the Presence of Defects

The goal of software testing is to make the software fail. Software testing reduces the presence of defects. Software testing talks about the presence of defects and doesn't talk about the absence of defects. Software testing can ensure that defects are present but it can not prove that software is defect-free. Even multiple tests can never ensure that software is 100% bug-free. Testing can reduce the number of defects but not remove all defects.

2. Exhaustive Testing is not Possible

It is the process of testing the functionality of the software in all possible inputs (valid or invalid) and pre-conditions is known as exhaustive testing. Exhaustive testing is impossible means the software can never test at every test case. It can test only some test cases and assume that the software is correct and it will produce the correct output in every test case. If the software will test every test case then it will take more cost, effort, etc., which is impractical.

3. Early Testing

To find the defect in the software, early test activity shall be started. The defect detected in the early phases of SDLC will be very less expensive. For better performance of software, software testing will start at the initial phase i.e. testing will perform at the requirement analysis phase.

4. Defect Clustering

In a project, a small number of modules can contain most of the defects. The Pareto Principle for software testing states that 80% of software defects come from 20% of modules.

5. Pesticide Paradox

Repeating the same test cases, again and again, will not find new bugs. So it is necessary to review the test cases and add or update test cases to find new bugs.

6. Testing is Context-Dependent

The testing approach depends on the context of the software developed. Different types of software need to perform different types of testing. For example, The testing of the e-commerce site is different from the testing of the Android application.

7. Absence of Errors Fallacy

If a built software is 99% bug-free but does not follow the user requirement then it is unusable. It is not only necessary that software is 99% bug-free but it is also mandatory to fulfill all the customer requirements.

Goals of Software Testing

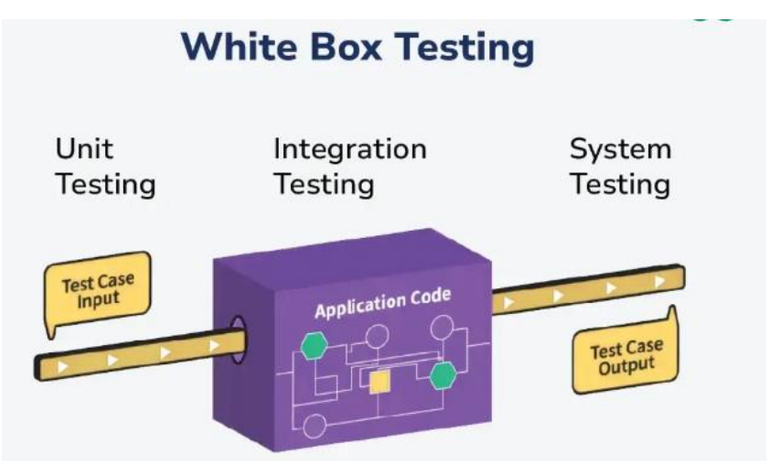
The main goals of software testing are designed to ensure the delivery of a high-quality software product that meets the needs of its users and stakeholders. Here are five important goals of software testing:

1. **Ensure Software Quality**: A fundamental goal of software testing is to ensure that the software meets the required quality standards and specifications. The quality involves the different characteristics or attributes, such as functionality, reliability, usability, efficiency, maintainability, and portability. Testing is done to prove these attributes of the software against a specified set of conditions in which they would be the software to perform properly.
2. **Detection and identification of defects**: the key objective is to make sure that detection and identification of defects, bugs, and errors of software are done in an early and effective manner.
3. **Reduced Development Costs:** In software development, most of the cost of rectifying software defects is in the identification of the software defects, and the earlier this is done, the cheaper it is. The cost of fixing a defect increases significantly with the time it has been detected, especially after deployment. Software testing helps minimize those costs by identifying the problem and dealing with it at an early stage.
4. **Verify Compliance with Requirements**: Software testing verifies that the software meets all specified requirements and complies with industry standards, legal regulations, and user expectations. This includes both functional requirements (what the software does) and non-functional requirements (how the software performs under certain conditions).
5. **Facilitate User Acceptance**: A third major goal of software testing, which is as important as the rest, is facilitating user acceptance. This includes subjecting software to various user-based tests to ensure it is intuitive, user-friendly, and capable of efficiently carrying out the desired tasks. This is actually a critical focus area of user acceptance testing (UAT) by the team to ensure that the software will, at the end of the day, meet the desires and expectations of the users.

White-Box Testing/Structural Testing

White box testing techniques analyze the internal structures the used data structures, internal design, code structure, and the working of the software rather than just the functionality as in black box testing. It is also called glass box testing clear box testing or structural testing. White Box Testing is also known as transparent testing or open box testing.

White box testing is a software testing technique that involves testing the internal structure and workings of a software application . The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.



White box testing uses detailed knowledge of a software's inner workings to create very specific test cases.

- **Path Checking:** Examines the different routes the program can take when it runs. Ensures that all decisions made by the program are correct, necessary, and efficient.
- **Output Validation:** Tests different inputs to see if the function gives the right output each time.
- **Security Testing:** Uses techniques like static code analysis to find and fix potential security issues in the software. Ensures the software is developed using secure practices.
- **Loop Testing:** Checks the loops in the program to make sure they work correctly and efficiently. Ensures that loops handle variables properly within their scope.
- **Data Flow Testing:** Follows the path of variables through the program to ensure they are declared, initialized, used, and manipulated correctly.

Types Of White Box Testing

Unit Testing

- Checks if each part or function of the application works correctly.
- Ensures the application meets design requirements during development.

Integration Testing

- Examines how different parts of the application work together.
- Done after unit testing to make sure components work well both alone and together.

Regression Testing

- Verifies that changes or updates don't break existing functionality.
- Ensures the application still passes all existing tests after updates.

Advantages of White Box Testing

1. **Thorough Testing** : White box testing is thorough as the entire code and structures are tested.
2. **Code Optimization:** It results in the optimization of code removing errors and helps in removing extra lines of code.
3. **Early Detection of Defects:** It can start at an earlier stage as it doesn't require any interface as in the case of black box testing.
4. **Integration with SDLC:** White box testing can be easily started in Software Development Life Cycle.
5. **Detection of Complex Defects:** Testers can identify defects that cannot be detected through other testing techniques.
6. **Comprehensive Test Cases:** Testers can create more comprehensive and effective test cases that cover all code paths.
7. Testers can ensure that the code meets coding standards and is optimized for performance.

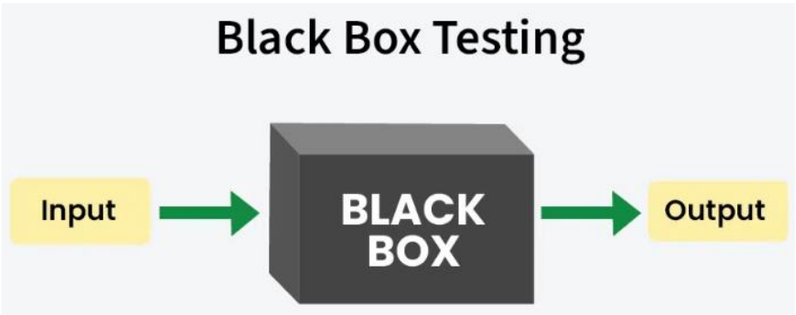
Disadvantages of White Box Testing

1. **Programming Knowledge and Source Code Access:** Testers need to have programming knowledge and access to the source code to perform tests.
2. **Overemphasis on Internal Workings:** Testers may focus too much on the internal workings of the software and may miss external issues.
3. **Bias in Testing:** Testers may have a biased view of the software since they are familiar with its internal workings.
4. **Test Case Overhead:** Redesigning code and rewriting code needs test cases to be written again.
5. **Dependency on Tester Expertise:** Testers are required to have in-depth knowledge of the code and programming language as opposed to black-box testing.
6. **Inability to Detect Missing Functionalities:** Missing functionalities cannot be detected as the code that exists is tested.
7. **Increased Production Errors:** High chances of errors in production.

Black-Box Testing /Functional.

Black Box Testing is an important part of making sure software works as it should. Instead of exploring the code, testers check how the software behaves from the outside, just like users would. This helps catch any issues or bugs that might affect how the software works.

Black-box testing is a type of software testing in which the tester is not concerned with the software's internal knowledge or implementation details but rather focuses on validating the functionality based on the provided specifications or requirements.



Types Of Black Box Testing

Functional Testing

- Functional testing is defined as a type of testing that verifies that each function of the software application works in conformance with the requirement and specification.
- This testing is not concerned with the source code of the application. Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.
- This testing focuses on checking the user interface, APIs, database, security, client or server application, and functionality of the Application Under Test. Functional testing can be manual or automated. It determines the system's software functional requirements.

Regression Testing

- Regression Testing is the process of testing the modified parts of the code and the parts that might get affected due to the modifications to ensure that no new errors have been introduced in the software after the modifications have been made.
- Regression means the return of something and in the software field, it refers to the return of a bug. It ensures that the newly added code is compatible with the existing code.
- In other words, a new software update has no impact on the functionality of the software. This is carried out after a system maintenance operation and upgrades.

Non-functional Testing

- Non-functional testing is a software testing technique that checks the non-functional attributes of the system.
- It is defined as a type of software testing to check non-functional aspects of a software application.
- It is designed to test the readiness of a system as per nonfunctional parameters which are never addressed by functional testing.
- It is as important as functional testing.
- It is also known as NFT. This testing is not functional testing of software. It focuses on the software's performance, usability, and scalability.

Advantages of Black Box Testing

- The tester does not need to have more functional knowledge or programming skills to implement the Black Box Testing.
- It is efficient for implementing the tests in the larger system.
- Tests are executed from the user's or client's point of view.
- Test cases are easily reproducible.
- It is used to find the ambiguity and contradictions in the functional specifications.

Disadvantages of Black Box Testing

- There is a possibility of repeating the same tests while implementing the testing process.
- Without clear functional specifications, test cases are difficult to implement.
- It is difficult to execute the test cases because of complex inputs at different stages of testing.
- Sometimes, the reason for the test failure cannot be detected.
- Some programs in the application are not tested.
- It does not reveal the errors in the control structure.
- Working with a large sample space of inputs can be exhaustive and consumes a lot of time.